

Agent Traffic Flow Management

Forecasting and Admission Control for LLM Agent Fleets

Alexander Aghili

Research report · September 27, 2026

Abstract

Large language model (LLM) agents alternate between inference and external tool execution. A serving system can therefore observe sessions that are likely to generate future requests before those requests arrive. We investigate whether this information improves aggregate demand forecasts and whether acting on the forecasts benefits latency-sensitive sessions. Agent Traffic Flow Management (ATFM) combines a session-state demand model, a tool-progress sidecar, and an admission proxy that can defer background requests. Across two public coding-agent corpora, the ratio of Kalman-baseline loss to elapsed-time-model loss ranges from 1.96 to 5.43 at five-minute horizons and from 2.73 to 5.09 at fifteen-minute horizons, using the 0.9-quantile pinball loss averaged over three seeds. In a separate collection of long-running tools, progress information reduces mean pinball loss from 24.9 to 8.9 seconds. Closed-loop simulation exposes a conditional benefit: in a loaded regime with short interactive tools, forecast-based holds improve session-weighted service-level-objective attainment by 9.3 percentage points while increasing mean background completion time by approximately 24%. A regime with long interactive tools and a shorter hold cap does not establish an advantage over rules-based admission, and a controlled cap ablation in the loaded regime removes the gain entirely: at a 60-second cap the forecast arms’ SLO differences from native span zero while their background costs remain, and a same-rule lookahead fed true demand does no better. These findings support session-aware forecasting but show that the measured policy gain depends on long deferral of background work; they do not isolate a benefit attributable to forecast accuracy. A revised capacity rule reduces unnecessary deferral but still does not establish an SLO gain. Serving results remain conditional on an unvalidated simulator and require hardware evaluation.

Keywords: LLM serving; agent workloads; probabilistic forecasting; admission control; KV cache.

1 Introduction

An LLM agent typically executes a sequence of model calls and external actions: a generated response invokes a tool, the tool produces a result, and that result becomes part of a subsequent prompt. While the tool runs, the session does not require inference compute, but its retained key-value (KV) cache may reduce the cost of resumption. Concurrent agents consequently create a scheduling problem that spans both requests already queued at the engine and sessions executing outside it.

Our premise is that the latter group provides useful information about future demand. A session’s current phase, elapsed tool time, context length, and available progress reports constrain when it may next request inference. This information is particularly relevant when many active sessions share a serving pool. It is less useful when new sessions dominate arrivals, external actions are unobservable, or uncertainty in return times overwhelms the controller’s decision horizon.

Figure 1 provides a running example. At the forecast time, three sessions are outside the model server. Two return during the next 30 seconds, while the third remains in its tool phase. The server has not received those two requests yet, but their elapsed execution and context sizes already provide evidence about future load. Our method formalizes this observation, first at the session level and then across the fleet.

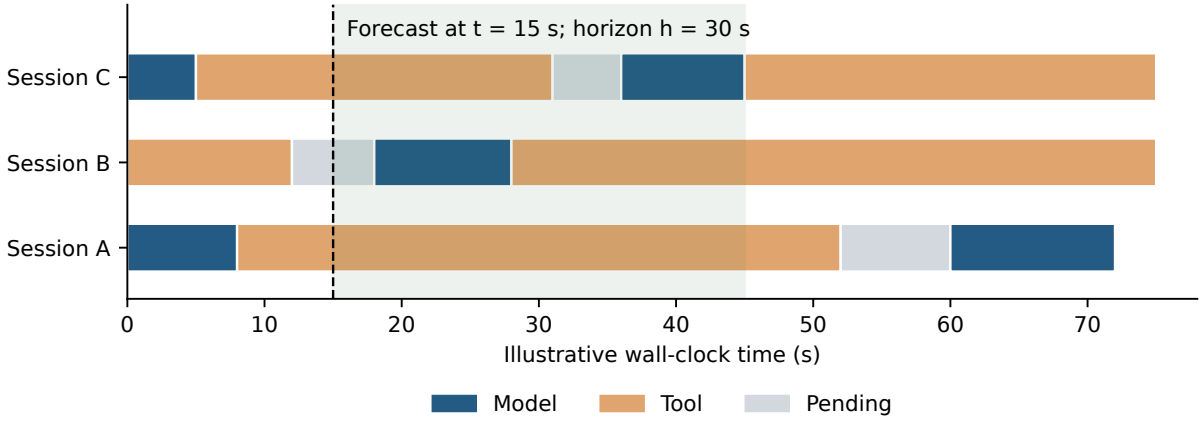


Figure 1: Illustrative agent timelines, not measured traces. Blue denotes model execution, orange tool execution, and gray a pending gap. The dashed line is the observation time; the shaded interval is the forecast horizon. Sessions B and C next request inference inside this horizon; A returns later.

ATFM maintains a *demand board* that aggregates per-session return-time distributions into forecasts of prompt-token and KV-block demand. An admission proxy uses these forecasts to delay deferrable work when interactive demand is expected to exceed available capacity. The forecast and admission decision are separate components; the implemented controller is a heuristic, without an optimality or service-level guarantee.

This report makes three contributions. First, it defines and implements a common forecast interface that separates the value of request history, elapsed tool time, and live progress. Second, it evaluates these information sources using held-out coding-agent traces and instrumented tool executions. Third, it measures the latency and background-completion trade-off of forecast-based holds in closed-loop simulation, including a no-hold ablation and a regime in which the policy does not outperform a simpler alternative. Forecast evaluation and policy evaluation are kept distinct because changing admission times changes subsequent agent arrivals.

The evidence addresses three research questions: **H1a**, whether in-flight session state improves fleet-demand forecasting over aggregate history; **H1b**, whether progress improves remaining-time prediction beyond elapsed time; and **H2**, whether forecast-based admission improves interactive latency at an explicitly measured cost to background work. The implementation and experiment records are described in Appendix A.

2 Background and Related Work

The running example raises two related questions: what state should remain available while an agent is away, and how should its eventual return affect scheduling? Prior systems address different parts of this problem. We first review the cache and program abstractions, then explain where ATFM’s forecast interface fits.

2.1 Why tool execution matters to the cache

A completed model turn does not necessarily end an agent session. Continuum studies the cost of discarding or offloading KV state between turns and retains selected caches with a time-to-live policy informed by reload cost and queueing delay [1]. Its motivating diagram (Figure 2) makes the return path explicit: the tool interval separates two inference intervals, and cache handling can add work or delay before the second begins. This motivates observing a session throughout its external execution.

End-of-turn KV Cache Eviction

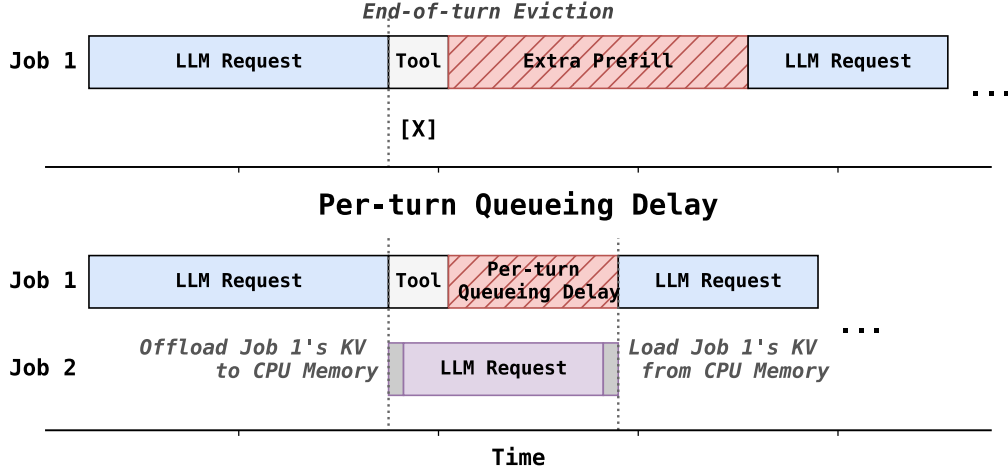


Figure 2: Cache handling across agent turns, reproduced from Li et al., *Continuum*, Figure 1, arXiv:2511.02230v7 [1]. The diagram contrasts end-of-turn eviction with CPU offloading and illustrates the resulting resumption overhead. Original diagram, unchanged except for scaling and SVG-to-PDF conversion; reproduced under the arXiv distribution’s CC BY 4.0 license.

2.2 From requests to programs

ThunderAgent represents multi-turn workflows as programs and coordinates inference and tool resources [2]. Figure 3 shows its program queue and backend resource metadata. The distinction between reasoning and acting matters because a program can remain active even when it has no request executing at the model server. This wider scheduling boundary motivates ATFM’s registry of in-flight sessions.

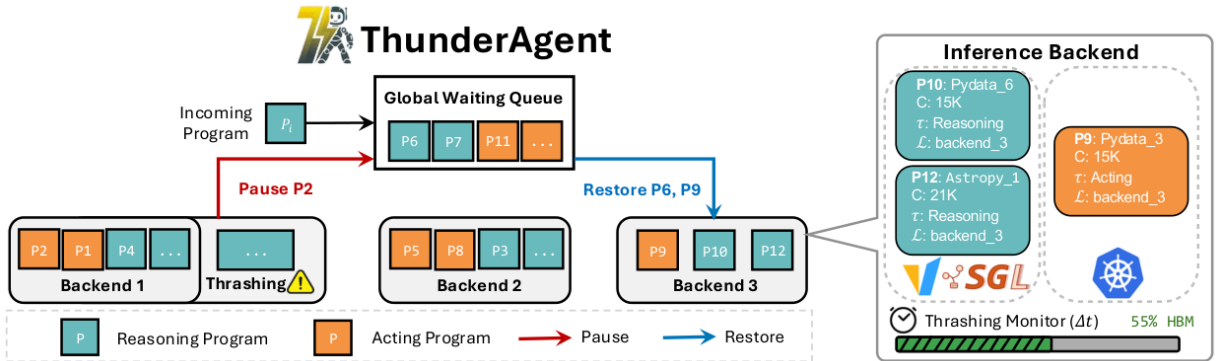


Figure 3: Program-aware resource management, reproduced from Kang et al., *ThunderAgent*, Figure 3, arXiv:2602.13692v3 [2]. Programs can be paused or restored according to backend state. Original diagram, unchanged except for scaling; reproduced under CC BY 4.0. Terminology and architecture in this diagram belong to ThunderAgent.

ATFM takes a complementary measurement perspective: it estimates aggregate first-resumption demand over explicit horizons and tests admission decisions driven by that estimate. Its history predictor and working-set policy are local baselines, not reproductions of Continuum or ThunderAgent. Their results cannot be interpreted as comparisons against either complete published system.

2.3 Progress signals and reactive alternatives

Liu et al. recover live tool progress for cache-management decisions [3], motivating the information source studied in H1b. ATFM does not claim novelty for extracting progress; it studies how session predictions aggregate and when a fleet controller benefits from them. ConServe instead emphasizes conversation-level observation and disaggregated scheduling [4]. This alternative motivates a rules-only baseline: a controller may benefit from session context without needing accurate duration predictions.

2.4 Infrastructure and workload evidence

NVIDIA Dynamo provides request hints, scheduling, and KV-aware serving infrastructure [5]. ATFM integrates a proxy with Dynamo’s Mocker path, while the policy experiments use a separate local engine model. DSec demonstrates stateful sandbox execution decoupled from GPU training [6]. Its reported scale—approximately three million sandboxes per day and more than 380,000 concurrent sandboxes—motivates tracking external execution, but does not establish demand predictability. TraceLab [7] and the SemiAnalysis AgentX corpus [8] provide the real session traces used here.

These systems establish the relevance of state outside the serving queue. The next section defines precisely what ATFM predicts from that state; Section 4 then describes how the proxy consumes the prediction.

3 Demand Model and Forecasting Method

The construction proceeds in three steps: define the demand target, estimate each session’s next return, and aggregate those estimates into a fleet distribution. Figure 4 summarizes the information flow. Throughout, a forecast describes uncertainty about the next request; it is not a reservation of serving capacity. Appendix C maps the mathematical statements to code and to the technical plan, distinguishing implemented rules from explanatory identities and proposed extensions.

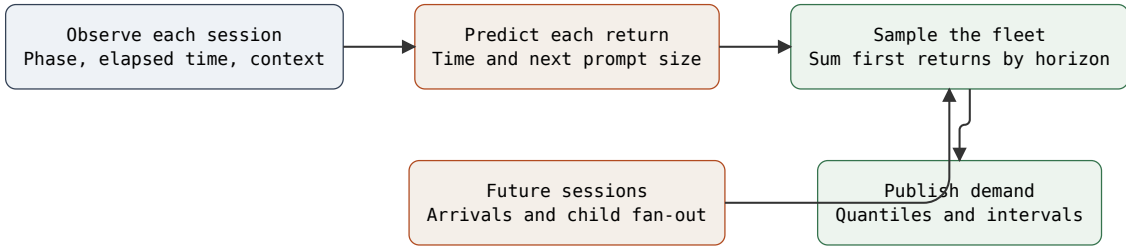


Figure 4: From observed session state to a distribution of fleet demand. Each Monte Carlo draw combines predicted first returns with modeled future-session demand. Quantiles are computed after aggregation, preserving the distinction between individual return uncertainty and aggregate load.

3.1 Forecast target

At time t , let $A_c(t)$ denote known sessions in class c , where the classes are interactive and background. For session i , let $R_i(t)$ be the time until its next model request and L_i the input-token count of that request. With block size $b = 16$ tokens, the KV-demand forecast is

$$D_c^{KV}(t, h) = \sum_{i \in A_c(t)} \mathbf{1}[R_i(t) \leq h] \left\lceil \frac{L_i}{b} \right\rceil + U_c^{KV}(t, h). \quad (1)$$

The term U represents demand from future sessions, including modeled child sessions and exogenous arrivals. Replacing $\lceil L_i/b \rceil$ with L_i gives the prompt-token target. Each session contributes only its first request in a horizon. Thus, these targets measure first-resumption demand, not all requests in the interval, instantaneous resident KV occupancy, or cache-miss-adjusted prefill work. The distinction matters because retention, prefix sharing, and repeated calls can change actual memory and compute requirements.

Known context length informs the next prompt size, but the size increment from tool output must be predicted. The live proxy also uses a character-count approximation when token counts are unavailable. Context size is therefore not assumed to be known exactly in deployment.

For clarity, the main quantities are collected in Table 1. Let $\mathcal{I}_t$ denote all information available by time t , and write $B_i = \lceil L_i/b \rceil$. Linearity of expectation gives

$$\mathbb{E}[D_c^{\text{KV}}(t, h) \mid \mathcal{I}_t] = \sum_{i \in A_c(t)} \mathbb{E}[B_i \mathbf{1}[R_i(t) \leq h] \mid \mathcal{I}_t] + \mathbb{E}[U_c^{\text{KV}}(t, h) \mid \mathcal{I}_t]. \quad (2)$$

If each B_i is fixed, its term simplifies to $B_i p_i(h)$, where $p_i(h) = \Pr(R_i(t) \leq h \mid \mathcal{I}_t)$. This identity does not require independent return times. For example, two known sessions of 100 and 200 blocks with return probabilities 0.8 and 0.3 contribute an expected $100(0.8) + 200(0.3) = 140$ blocks, before future-session demand. The expectation alone does not determine an upper quantile: dependence and dispersion also matter.

Table 1: Core notation. Times are measured in seconds and token counts in tokens unless stated otherwise.

Symbol	Meaning
$t, h, c, A_c(t)$	Observation time, forecast horizon, workload class, and known sessions of that class.
$R_i(t), L_i, B_i$	Time to the next model request, its prompt length, and its demand in KV blocks.
$b, D_c^{\text{KV}}, U_c^{\text{KV}}$	Tokens per block (16), total first-resumption demand, and the future-session contribution, both in blocks.
$T, e, r, S(u)$	Total tool duration, elapsed duration, candidate remaining duration, and survival probability $S(u) = \Pr(T > u)$.
$\mathcal{I}_t, M, \tau$	Information observed by time t , number of Monte Carlo draws, and quantile level.

3.2 State and predictor hierarchy

The registry distinguishes model execution, tool execution, and a pending phase between tool completion and the next request. Events carry session identity, workload class, timestamps, tool identity, backend identity, and optional progress or data signals. Trace adapters map these events to a common table with one row per model call and its associated tool phase. Pending gaps are modeled separately; treating every completed tool as an immediate request would systematically misrepresent human and harness delays.

Table 2: Forecast models and the information available to each. All session models use the same aggregation procedure.

Model	Information and prediction rule
B0: persistence	Point forecast equal to the most recently realized demand window.
B1: Kalman	Local linear trend over realized demand; predictive Gaussian samples clipped at zero.
B2: history	Unconditional empirical tool-duration samples, minus elapsed time and clipped at zero.
M1: survival	Tool-duration samples conditioned on the tool still running at the current elapsed time.
M2: progress	M1 augmented with progress-rate estimation and a learned mapping from reported progress to elapsed-time fraction.
M3: backend	Progress prediction augmented with a shared backend slowdown factor; exploratory only.

B2 uses elapsed time in the final subtraction but does not condition its duration distribution on survival. For tool duration T and elapsed time e , the idealized survival model instead uses

$$\Pr(T - e > r \mid T > e) = \frac{S(e+r)}{S(e)}, \quad r \geq 0, \quad (3)$$

where S is the survival function.

Conditioning removes durations incompatible with a tool that is still running. In the illustrative history $T \in \{10, 20, 40, 80\}$ seconds with equal weights, a tool observed at $e = 30$ seconds can have remaining time 10 or 50 seconds under M1, each with probability one half. B2 instead maps the two already-finished historical durations to zero, placing half its mass on immediate completion (Figure 5). The empirical M1 conditional CDF, before fallback and gap handling, is

$$\hat{F}_{R|e}(r) = \frac{\sum_{j=1}^{n_T} \mathbf{1}[e < T_j \leq e+r]}{\sum_{j=1}^{n_T} \mathbf{1}[T_j > e]}, \quad r \geq 0, \quad (4)$$

where T_j is a historical duration and n_T is the number of available durations. A zero denominator requires the implementation's finite-sample fallback; Equation (4) alone is undefined in that case.

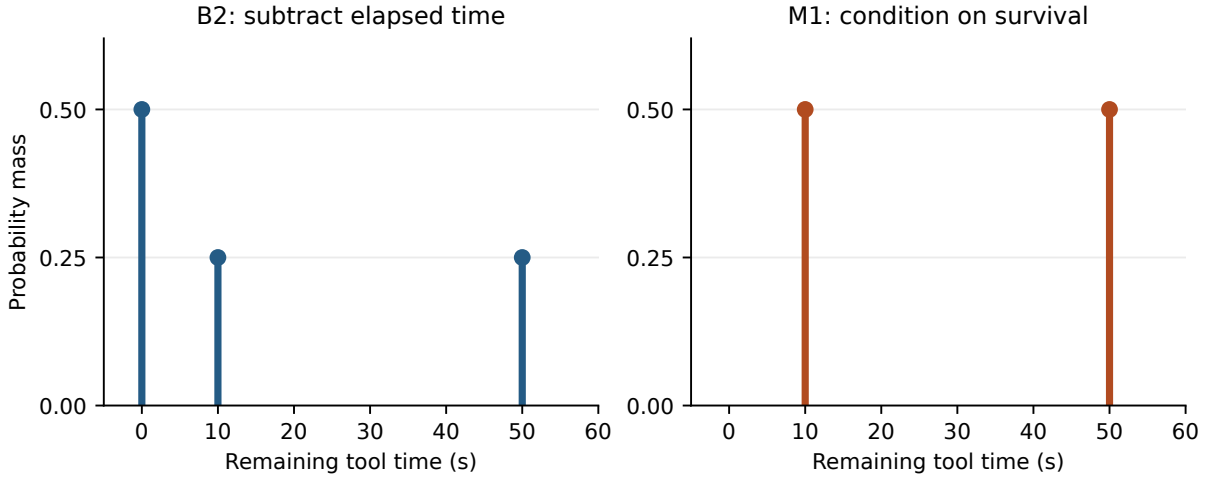


Figure 5: Illustrative remaining-time distributions for the same four historical durations at elapsed time 30 s. Vertical stems are probability masses, not densities. Conditioning excludes completed historical cases instead of interpreting them as evidence of immediate completion.

The implementation uses empirical duration distributions with finite-sample fallbacks, adds a tool-to-request gap, and assigns probability mass to sessions that do not return. M2 combines a rate filter with empirical progress curves because a tool's reported fraction complete need not be linear in wall-clock time. When progress is absent, M2 falls back to the elapsed-time model.

To see why a progress curve is needed, let $p \in (0, 1]$ be the reported fraction of work completed and $g(p)$ the learned fraction of wall-clock duration elapsed at that report. If the report arrives at elapsed time e_ℓ , the curve-based central estimate at current elapsed time e is

$$\hat{T} = \frac{e_\ell}{g(p)}, \quad \hat{R}_{\text{tool}} = \max\left(\frac{e_\ell}{g(p)} - e, 0\right). \quad (5)$$

Here $\hat{T}$ estimates total tool duration, and $\hat{R}_{\text{tool}}$ excludes the subsequent pending gap. The implementation adds sampled multiplicative dispersion and residual variation, so this central estimate is not the full predictive distribution. When no learned curve is available, the rate-based branch samples a work rate v and estimates remaining time from $(W - w)/v$, where W and w are total and completed work units, with adjustment for report age. Figure 6 shows how two different mappings of reported progress change the estimate even with identical observed elapsed time.

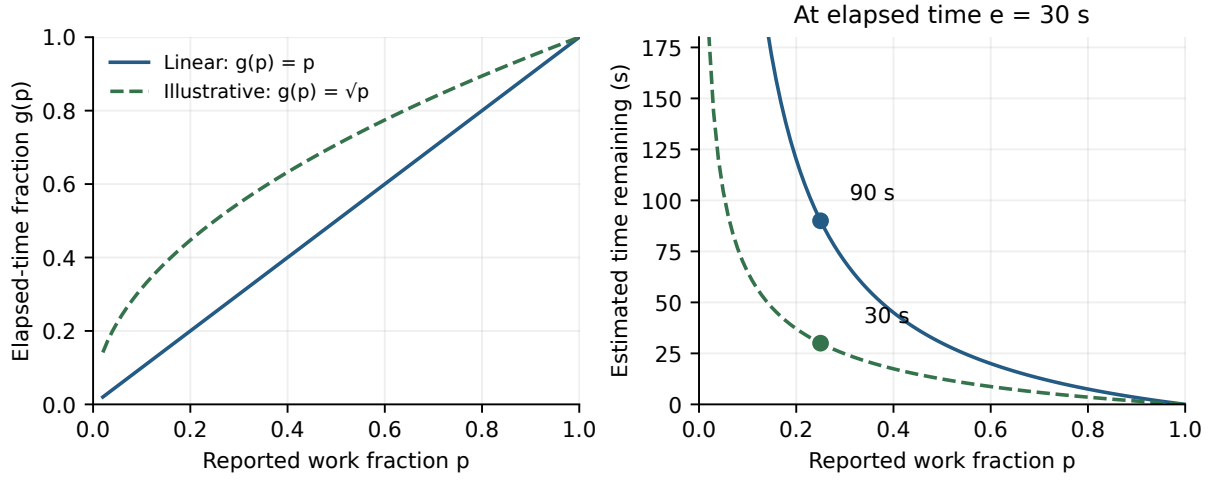


Figure 6: Illustrative progress mappings, not fitted experimental curves. Left: reported work fraction can differ from elapsed-time fraction. Right: with a fresh report at elapsed time 30 s and reported progress 0.25, the linear assumption predicts 90 s remaining; the illustrative square-root curve predicts 30 s. The discrepancy motivates learning tool-specific mappings.

3.3 A probability model for the next request

The preceding examples concern tool completion, whereas the forecast target is the next model request. These events differ when a completed tool is followed by a harness delay or the session never resumes. For a session currently executing a tool, let $Z_i \in \{0, 1\}$ indicate whether it will return, $\zeta_i = \Pr(Z_i = 1 \mid \mathcal{I}_t)$, $Q_i \geq 0$ the remaining tool duration, and $G_i \geq 0$ the subsequent pending gap. A useful representation is

$$R_i(t) = \begin{cases} Q_i + G_i, & Z_i = 1, \\ \infty, & Z_i = 0. \end{cases} \quad (6)$$

Thus a missing return is probability mass at infinity, not an arbitrarily long finite tool time. Under conditional independence of Q_i and G_i among returning sessions,

$$p_i(h) = \Pr(R_i(t) \leq h \mid \mathcal{I}_t) = \zeta_i \int_{[0, h]} F_{Q_i}(h - g) dF_{G_i}(g). \quad (7)$$

The CDFs F_{Q_i} and F_{G_i} are conditioned on the observed state and on return; this conditioning is suppressed to keep the expression readable. The integral includes a possible atom at zero gap. As $h \rightarrow \infty$, the finite-time CDF approaches ζ_i , provided both component durations are finite. The implementation samples tool remainder and gap separately and then applies a learned per-tool no-return probability in its tool-running branch. This is a modeling factorization, not evidence that the real components are independent. Pending and model-running states use their own phase-specific rules; Equation (6) is not a claim that every branch currently estimates a separate termination probability.

Elapsed time can either shorten or lengthen the remaining wait. For a continuous duration distribution with density f_T , define the completion hazard $a(u) = f_T(u)/S(u)$, wherever $S(u) > 0$. It measures instantaneous completion probability per unit time conditional on still running. Equation (3) can then be written

$$\Pr(Q > r \mid T > e) = \exp \left[- \int_e^{e+r} a(u) du \right]. \quad (8)$$

A constant hazard gives a memoryless remaining duration; increasing and decreasing hazards produce different responses to elapsed time. For the illustrative Weibull survival function $S(u) = \exp[-(u/\theta)^k]$, the conditional median is

$$q_{0.5}(Q \mid T > e) = \theta \left[(e/\theta)^k + \log 2 \right]^{1/k} - e, \quad (9)$$

where $\theta > 0$ is a time scale and $k > 0$ a shape parameter. Figure 7 shows why subtracting elapsed time from an unconditional duration is inadequate: surviving a long time can itself be evidence of a long remaining wait. The Weibull examples explain conditioning; M1 uses empirical durations with fallbacks, not these fitted parametric curves.

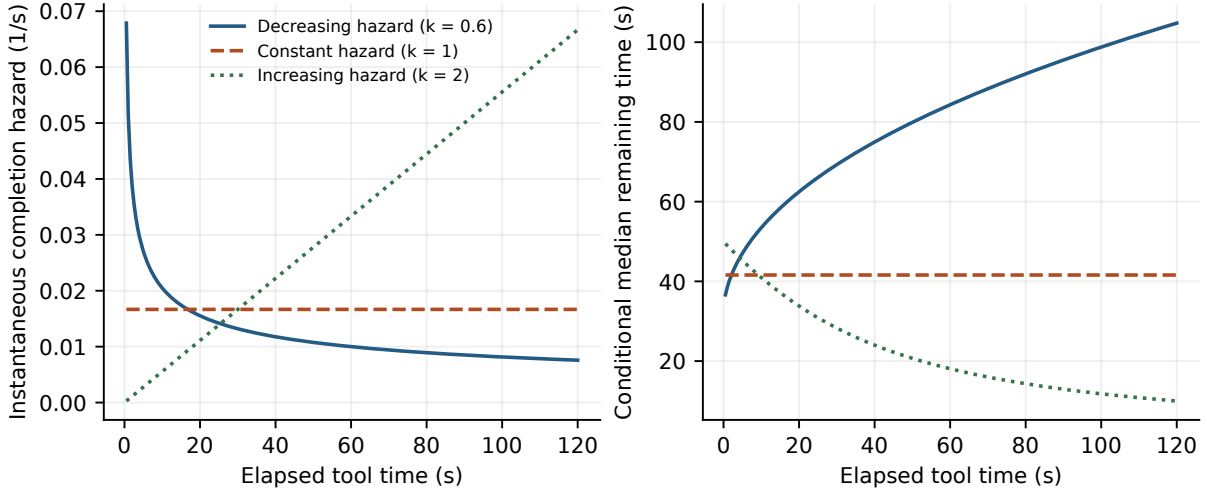


Figure 7: Illustrative duration models with common scale $\theta = 60$ s and shapes $k \in \{0.6, 1, 2\}$. Left: conditional completion hazard. Right: median remaining duration after survival to the elapsed time. Equal scale parameters do not imply equal means. These analytic examples are not fits to the tool corpus.

Progress updates uncertainty about execution rate. The rate-based branch of M2 has a Gamma-update interpretation. With a fixed shape–rate prior $V \sim \text{Gamma}(a_0, b_0)$, a constant-rate Poisson working model for completed units over positive intervals is $\Delta W_j \mid V \sim \text{Poisson}(V\Delta t_j)$. Multiplying likelihood by prior gives

$$p(v \mid \mathcal{P}) \propto v^{a_0-1+\sum_j \Delta W_j} \exp[-v(b_0 + \sum_j \Delta t_j)], \quad (10)$$

$$V \mid \mathcal{P} \sim \text{Gamma}(a, b), \quad a = a_0 + \sum_j \Delta W_j, \quad b = b_0 + \sum_j \Delta t_j. \quad (11)$$

Here $\mathcal{P}$ denotes the observed progress increments, v is work units per second, a is dimensionless, and b has units of seconds when work is counted in units. The posterior mean is a/b , and its coefficient of variation is $1/\sqrt{a}$. This conjugate calculation motivates the implemented shape/rate update; the actual default prior rate is estimated from the same progress stream, so the implementation is an empirical-Bayes approximation rather than the fixed-prior experiment in Figure 8. Fractional counters likewise need not obey a literal Poisson count model.

For remaining work $w_* > 0$ at a fresh report, the idealized completion time is $Q = w_*/V$. A change of variables yields

$$f_Q(r) = f_V(w_*/r) \frac{w_*}{r^2}, \quad r > 0, \quad \mathbb{E}[Q] = \frac{w_* b}{a-1} \quad (a > 1). \quad (12)$$

Consequently, using $w_*/\mathbb{E}[V] = w_* b/a$ is not the same as the mean remaining time: uncertainty in a reciprocal increases the latter. The implementation samples the rate, then applies report-age and residual adjustments. Its learned progress-curve branch remains preferred when an appropriate curve is available.

3.4 Aggregation and calibration

The demand board draws Monte Carlo return times and next-call sizes, then sums their contributions using Equation (1). It models one level of child-session fan-out and a Poisson process for exogenous arrivals.

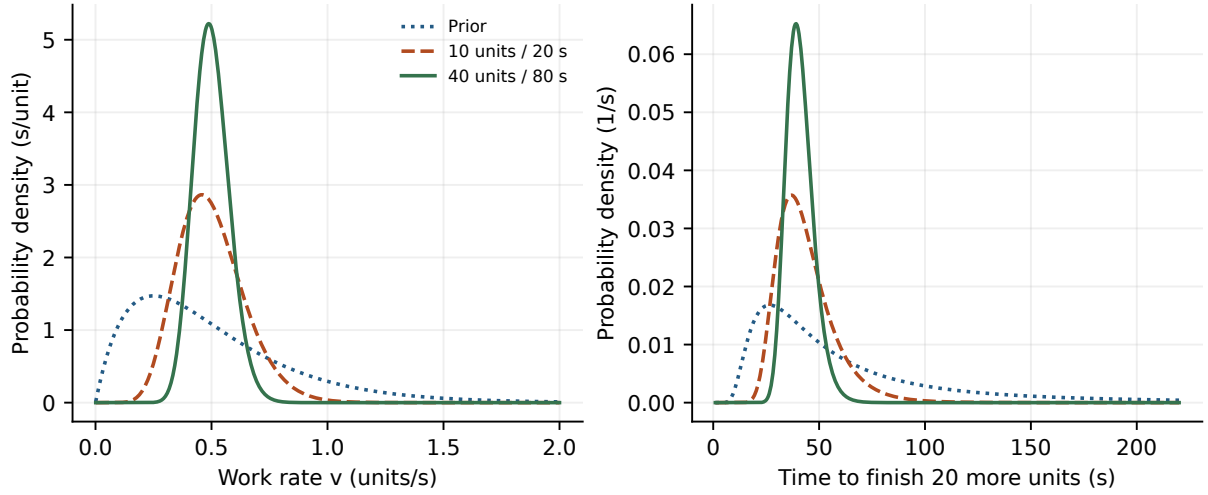


Figure 8: Illustrative fixed-prior rate update, with Gamma shape 2 and rate 4 s. Ten completed units in 20 s and forty in 80 s imply the same empirical rate but different uncertainty. Left: prior and posterior rate densities. Right: the induced remaining-time densities for a common 20 units of work still to do. These are comparative scenarios with the same remaining work, not successive states of one fixed-size job.

The arrival rate is estimated from recent session starts; first-call sizes are sampled from training data. History-only predictors receive a realized demand window only after its full horizon has elapsed, preventing future-window leakage.

Formally, for draw $m = 1, \dots, M$, the board computes

$$D_c^{(m)}(t, h) = \sum_{i \in A_c(t)} \mathbf{1}[R_i^{(m)}(t) \leq h] B_i^{(m)} + U_c^{(m)}(t, h), \quad (13)$$

$$\hat{F}_{c,t,h}(d) = \frac{1}{M} \sum_{m=1}^M \mathbf{1}[D_c^{(m)}(t, h) \leq d], \quad \hat{q}_\tau = \inf\{d : \hat{F}_{c,t,h}(d) \geq \tau\}. \quad (14)$$

The superscript (m) identifies a sampled scenario, d is a candidate block demand, and $\hat{q}_\tau$ is the empirical demand quantile. Quantiles are taken over fleet sums, rather than summing per-session quantiles. For the exogenous part alone, the model draws $N_h \sim \text{Poisson}(\hat{\lambda}h)$ new sessions and sums N_h sampled first-call sizes; $\hat{\lambda}$ is the estimated start rate in sessions per second. Child-session demand is modeled separately within $U_c^{(m)}$.

Forecast dispersion can be calibrated by fitting a variance-inflation factor separately for each class and horizon on a held-out portion of the training data. Calibration uses an overlaid fleet at the evaluation arrival rate. This adjustment changes interval width, but it does not guarantee calibration under model bias, dependence between sessions, or workload shift. M3 is implemented but is not a basis for the main claims: an earlier synthetic slowdown experiment showed inconsistent gains across horizons.

3.5 Fleet uncertainty, dependence, and overload probability

Aggregation adds a second kind of uncertainty: knowing each session's marginal return probability does not specify whether sessions return together. For fixed known block sizes, define $I_i(h) = \mathbf{1}[R_i(t) \leq h]$, $X(h) = \sum_i B_i I_i(h)$, and $p_{ij}(h) = \Pr(I_i(h) = 1, I_j(h) = 1 \mid \mathcal{I}_t)$. With conditioning on $\mathcal{I}_t$ understood,

$$\text{Var}(X) = \sum_i B_i^2 p_i (1 - p_i) + 2 \sum_{i < j} B_i B_j \{p_{ij} - p_i p_j\}. \quad (15)$$

The first term is the sum of individual uncertainties. The second measures synchronization: shared stalls or release waves can increase fleet variance without changing any individual marginal probability. For random

prompt sizes, the general expression is $\sum_i \text{Var}(B_i I_i) + 2 \sum_{i < j} \text{Cov}(B_i I_i, B_j I_j)$; size–return dependence must also be specified. The main M1/M2 sampling path draws different sessions separately. M3 introduces a shared backend factor, but the main experiments do not validate its joint distribution.

For example, let two sessions each require 100 blocks and each return within the horizon with probability one half. Independent returns give total-demand probabilities (0.25, 0.50, 0.25) at (0, 100, 200) blocks. Perfectly synchronized returns instead give (0.50, 0, 0.50). Both distributions have expected demand 100 blocks; their variances are 5,000 and 10,000 blocks squared. At capacity 150 blocks, overload probabilities are 0.25 and 0.50 (Figure 9). Larger variance does not increase exceedance at every threshold: below 100 blocks the ordering reverses.

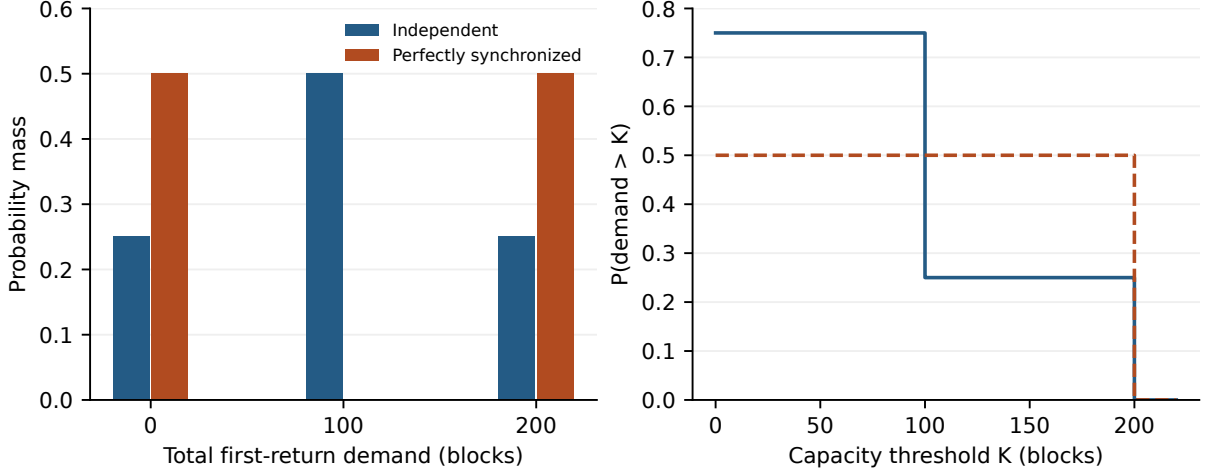


Figure 9: Exact two-session example with identical marginal return probabilities and different dependence. Left: fleet-demand probability mass. Right: exceedance probability as a function of a block threshold. Equal mean demand is insufficient for a capacity decision; the joint distribution matters. This is an analytical illustration, not an empirical estimate of session correlation.

Future sessions add a compound arrival process. For the exogenous component $U_{\text{exo}} = \sum_{j=1}^{N_h} V_j$, let $N_h \sim \text{Poisson}(\lambda h)$ and let V_j be independent, identically distributed first-call block sizes, independent of N_h , with mean μ_V and finite second moment. Then

$$\mathbb{E}[U_{\text{exo}}] = \lambda h \mu_V, \quad \text{Var}(U_{\text{exo}}) = \lambda h \mathbb{E}[V_j^2]. \quad (16)$$

The variance uses the second moment, not merely $\text{Var}(V_j)$, because the number of sessions is random. These identities describe the exogenous sampler at a fixed estimated rate; child-session fan-out is an additional component. If the rate itself were uncertain, Λ , total variance would become

$$\text{Var}(U_{\text{exo}}) = h \mathbb{E}[\Lambda] \mathbb{E}[V_j^2] + h^2 \text{Var}(\Lambda) \mu_V^2, \quad (17)$$

assuming size marks are independent of the rate. The current sampler plugs in a rate estimate rather than sampling Λ ; the second term identifies an omitted uncertainty source, not an implemented feature.

Figure 10 combines known-session returns and future arrivals in a small worked fleet. Known sessions can each contribute only once, while the expected exogenous contribution continues to grow with horizon. This helps explain why a useful short-horizon session signal need not determine long-horizon demand.

Demand risk is distinct from resource risk. For a block-demand threshold K , two summaries of the forecast are

$$\hat{p}_K = \frac{1}{M} \sum_{m=1}^M \mathbf{1}[D^{(m)} > K], \quad \hat{e}_K = \frac{1}{M} \sum_{m=1}^M (D^{(m)} - K)_+, \quad (18)$$

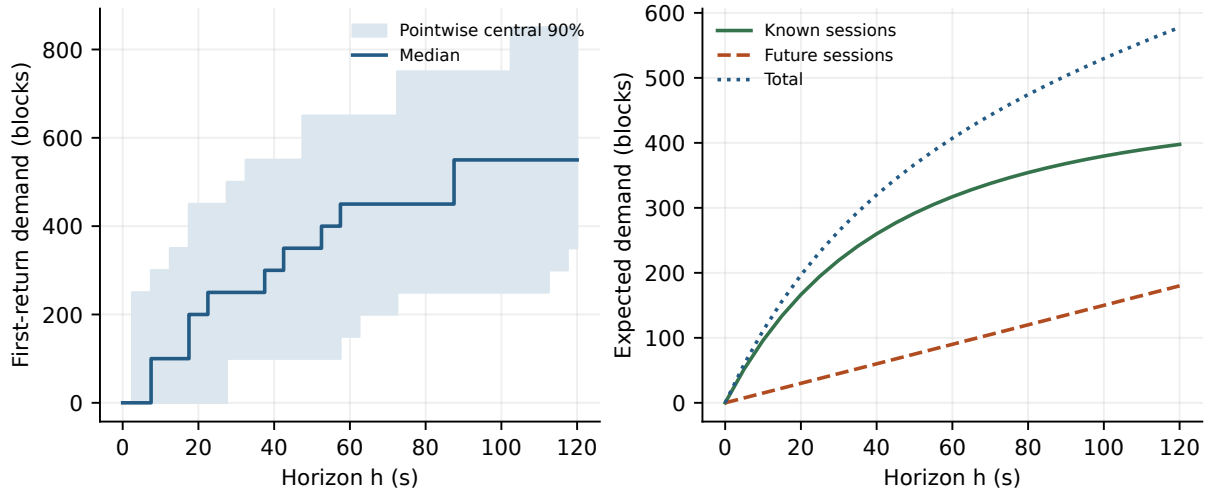


Figure 10: Illustrative fleet with three known sessions of 100, 150, and 200 blocks and independent exponential return times of mean 20, 40, and 80 s. Future sessions arrive at 0.015/s, each contributing 100 blocks; child fan-out is omitted. Left: median and pointwise central 90% band from 20,000 seeded draws. Right: analytic expectation decomposed into known and future sessions. The band is not a simultaneous guarantee over horizons, and these are not experiment traces.

where $(x)_+ = \max(x, 0)$. The first is estimated exceedance probability; the second is expected excess in blocks. An unlikely but large exceedance can have small $\hat{p}_K$ and substantial $\hat{e}_K$. For the synchronized example at $K = 150$, these values are 0.50 and 25 blocks, compared with 0.25 and 12.5 blocks for independent returns. Neither quantity is the probability of physical cache overflow unless the predicted demand target and threshold faithfully represent occupancy. This is precisely the approximation made by the hold rule in Section 4.3.

A further distinction is between marginal horizons and joint paths. The current exogenous sampler draws new totals separately for each horizon. Its individual horizon distributions can therefore be used for pointwise scoring, but matching draw indices across horizons need not produce a monotone cumulative-arrival path. The illustrative paths in Figure 10 use shared Poisson increments and are coherent by construction. A controller evaluating horizon-wide overload must similarly couple draws across time; pointwise quantiles alone do not supply that joint forecast.

4 System and Admission Policy

A demand distribution becomes operational only when it changes a decision. This section follows that path from sidecar observation to proxy admission, then makes explicit the capacity approximation used by the evaluated hold rule. The distinction between prediction and control will be important when interpreting the cap ablation.

4.1 Architecture and prototype

Figure 11 separates the request path from the measurement and control path. The sidecar observes tool output and emits structured events while preserving the subprocess result. The demand board consumes events and publishes forecast samples. The proxy maintains an admission window and applies ordering and hold decisions before forwarding requests to the serving engine. The Python prototype includes in-memory and JSONL event buses, a FastAPI proxy, a live board, corpus adapters, and a discrete-event simulator. Durable event transport, explicit hardware KV tier placement, and replica-floor control remain architectural extensions; simulator-only eviction ordering is implemented but not evaluated in this report.

The sidecar parses build percentages, test counters, generic work counters, and output-rate signals. Parser

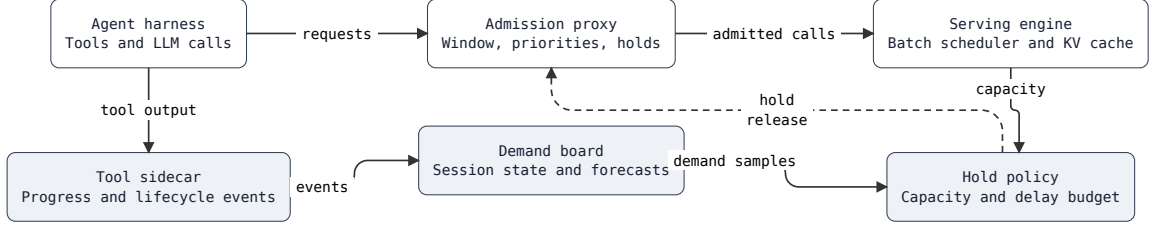


Figure 11: ATFM’s logical architecture. The upper row carries model requests; the lower row carries tool state, forecasts, and capacity information. The dashed connection denotes control of admission. The simulator exercises the feedback loop; the Dynamo Mocker experiment verifies request integration rather than real-GPU capacity feedback.

and event-publication failures are handled without changing tool output. The proxy accepts external hold directives and falls back to default predictions if the prediction service fails; this does not imply that an unavailable serving backend can be bypassed. The recorded Mocker test exercised three concurrent scripted sessions, a two-request admission window, and deadline-sensitive priority hints. It did not measure GPU memory behavior, model quality, or end-to-end controller effectiveness on hardware.

4.2 Ordering and admission

A global window bounds requests released to the engine. Waiting calls are ordered first by class and deadline tier, then by an index. Interactive calls receive tier 1, or tier 2 when estimated deadline slack is below a threshold; background calls receive tier 0. Within a tier, the index is

$$\pi_i = \frac{w_{c(i)} (1 + \beta \mathbb{E}[T_{next,i}])}{\mathbb{E}[S_i]}, \quad (19)$$

where $w_{c(i)}$ is the class weight, S_i is model service time, and $T_{next,i}$ is the next tool duration. The coefficient β controls the preference for serving sessions that will subsequently spend time outside the engine. Service time is estimated from prompt and output lengths using configured prefill and decode rates. Rules-only admission sets the next-tool term to zero. This index is a heuristic, not a Gittins-index solution or an optimal scheduling policy.

4.3 Forecast-based holds and their costs

The GDP-lite policy consults a 30-second forecast slot when a background call becomes eligible for admission. It holds the call if either the 0.9 quantile of interactive KV demand exceeds capacity not occupied by running requests, or an approximation to average interactive batch occupancy exceeds free batch slots. The occupancy approximation divides forecast prompt tokens by mean prompt size to estimate call count, then multiplies by expected service time and divides by slot duration. These tests approximate resource pressure; neither directly enforces a probabilistic bound on actual simultaneous occupancy.

Let $K_f(t)$ and $C_f(t)$ be currently free KV blocks and batch slots, $\bar{L}$ the estimated mean interactive prompt size, and $\bar{S}$ the estimated mean interactive model-service time. For slot length $h_s = 30$ seconds, the evaluated rule can be written as

$$\widehat{O}(t) = \frac{q_{0.9}(D_{\text{int}}^{\text{tok}}(t, h_s)) \bar{S}}{\max(\bar{L}, 1) h_s}, \quad (20)$$

$$H(t) = \mathbf{1} \left[q_{0.9}(D_{\text{int}}^{\text{KV}}(t, h_s)) > K_f(t) \vee \widehat{O}(t) > C_f(t) \right]. \quad (21)$$

The subscript int denotes interactive sessions, D^{tok} is the prompt-token target, $q_{0.9}$ is its forecast 90th percentile, and $H(t) = 1$ requests a background hold subject to its cap. $\widehat{O}$ has units of average occupied

batch slots. The numerator estimates call volume over the entire slot, whereas the free-capacity terms are instantaneous. This temporal mismatch explains why a persistently busy engine can trigger holds even when forecast refinements would not change the decision. Equation (21) describes the original rule; Section 6.6 evaluates a capacity-release revision.

Holds are reconsidered at expiry and capped relative to request arrival. Expiration of the policy hold does not bypass the admission window, so ordinary queueing can continue after the cap. The simulator separately records policy-hold duration, ordinary queueing, held-cache occupancy integrated over time, displacement evictions, and prefix recomputation. A delay limit consequently bounds imposed hold time, not total request latency or session completion time.

The completed loaded experiment uses a 600-second hold cap. The long-interactive-tool experiment uses 60 seconds. Where the admission window equals engine batch capacity, the window-based arms place their backlog at the proxy. The native arm queues at the engine. This queue placement is part of the treatment, and results do not isolate the effect of engine-side priority hints.

4.4 Control implementation and scaling

The live prototype and the policy simulations have distinct performance costs. The implementation revisions described here address live control overhead; they do not revise the reported simulation outcomes or establish hardware throughput.

Incremental event consumption. Previously, live JSONL draining reused the full-file offline reader, so each tick parsed and reapplied historical records. A per-consumer byte cursor now separates incremental consumption from full replay. With bounded event size, parsing costs $O(\Delta E)$ for newly appended events rather than $O(E)$ for the entire history; an unchanged log needs only a metadata check. A new board still replays from the beginning to reconstruct state. Complete malformed lines are counted and skipped, partial last lines are retried, and observed truncation or file replacement resets the cursor. Rotation requires coordination to avoid losing an unread old-file tail; unobserved copy-truncate/regrowth is unsupported. This cursor does not checkpoint model state or acknowledge successful downstream application. Crash-safe recovery still requires consistent state/log checkpoints and duplicate handling. Consumed arrival records are discarded while future timestamps are retained, preserving the existing exclusion of starts older than the previous board tick.

The design prioritizes event semantics before CPU kernels. Tests exercise repeated drains and board ticks, partial and malformed lines, independent readers, file replacement, observed truncation, and I/O retry. Component benchmarks and their source hashes accompany the development documentation; none measures GPU service capacity or end-to-end tail latency.

Batched control delivery. Previously each hold caused a serial HTTP request and a fresh queue scan. The controller now sends bounded batches of at most 1,024 holds. The proxy validates all records before applying a batch, discards expired updates, and performs one admission pass. For D directives, backlog Q , and batch bound B , the repeated list-scan term changes from $O(DQ)$ to $O(D + \lceil D/B \rceil Q)$, excluding release selection. Round trips decrease from D to $\lceil D/B \rceil$; payload work stays $O(D)$. Chunks are independent, not a durable whole-plan transaction. Updates retain their original meaning: they govern subsequent submissions and do not retime already queued requests. Versioned deltas and automatic retry are deferred because expiry renewal and restart synchronization require a separate protocol. The per-request board client also retrieves service and next-tool estimates in one request, with local fallback on unavailable results. CPU work still shares the board event loop; these changes do not establish request-tail isolation.

Indexed admission. The live queue previously scanned its pending list for every individual completion. It now maintains priority and FCFS ready heaps, delayed-release and promotion heaps, and request-identity dictionaries. Stable insertion sequences resolve ties; versioned records invalidate old priorities, and

periodic linear-time rebuilds bound stale heap storage. Ordinary release costs $O(\log Q)$ amortized, reducing individual backlog drainage from $O(Q^2)$ to $O(Q \log Q)$; simultaneous due events still require processing each affected request. Overflow transitions retain the original hold-dropping and FCFS policy and can cost $O(Q)$. A single wake-up timer tracks holds and promotions. Randomized differential tests compare release order and state with the former scan implementation. Diagnostic counts still scan the backlog. The later peer-rank histogram scans distinct indices, retaining a linear worst case, so this is not a logarithmic bound on the whole HTTP path. The simulator’s worker queue is a separate implementation.

Planner search and forecast sampling. GDP search now bounds candidate slots before allocation using the original delay comparison. For S sessions, F total slots, and W eligible slots, vector search costs $O(SW)$ plus $O(S \log F)$ interval-bound work, excluding threshold preparation and sorting. For larger windows, a lazily constructed segment tree stores per-resource minima of the exact threshold-plus-commitment quantity. A range is skipped when any resource cannot fit even at that minimum. Passing a range test does not imply joint feasibility, since resource minima can occur in different slots; leaves retain exact comparisons. Search therefore remains $O(F)$ in the worst case, with $O(\log F)$ commitment updates for fixed resource count. Small windows retain vector search, and open cases need not build the tree. Tests compare full directives and assignments with the original sample-scan planner, including floating-point boundaries and tenant-cap overrides.

Empirical forecast sampling uses integer-indexed draws instead of generic unweighted replacement choice. This preserves sampled values and random-stream consumption under the tested NumPy generators while reducing repeated small-call overhead. It does not change the $O(SMH)$ aggregation order for M draws and H horizons, nor introduce shared session samples. Cross-session batching, reuse of forecast draws for directive quantiles, and separation of control computation from request handling remain future work requiring statistical or state-ownership validation. The implementation changes do not imply a change in forecast quality.

Component measurements. Table 3 compares saved pre-change medians with the updated implementation on the same Intel Core Ultra 9 185H workstation. Each timing has one warmup and three repetitions; fixture setup is excluded. CPU affinity and background activity are uncontrolled, and baseline runs were taken separately. The JSONL fixture has no newly appended events; its initial replay still reads all records. Queue drainage signals individual completions without service time. The GDP cases are extreme stress fixtures, not the local ten-slot configuration. The forecast fixture uses homogeneous tool-running sessions, three horizons, 128 draws, and no exogenous arrivals or fan-out. GDP and forecast output hashes match their saved baselines exactly. These component costs cannot be summed into an end-to-end latency estimate; persistent queue indexes also add construction and memory costs not measured by the drainage fixture.

Table 3: Local component medians in seconds; independent synthetic workloads.

Component and workload	Before	After
Unchanged JSONL, 10^6 historical events	4.297	0.0000063
Single-slot drainage, 4,000 queued requests	1.974	0.00703
Saturated GDP, 10^5 sessions / 10^4 slots	7.322	1.691
Mixed GDP, 10^5 sessions / 10^4 slots	4.706	1.874
Saturated GDP, 10^6 sessions / 300 slots	15.910	7.477
Forecast, 10^5 sessions	4.120	3.785

Integrated load-test harness. A separate experiment package now runs the real proxy, board, and control loop against a bounded-concurrency fake model worker in independent localhost HTTP processes. Scheduled new-session arrivals remain independent of earlier sessions; each session makes sequential model calls separated by tool progress and completion events. A synchronized-completion variant exercises

bursts. The harness retains client and proxy traces, control-call timings, observed queue depths, snapshot age, process CPU/memory, event-loop lag, generator lateness, and incomplete-run outcomes. Its initial six cases completed 2,016 synthetic calls without request or control errors. Trace alignment exposed substantial pre-proxy delay in one burst case, so client transport timestamps were added before attributing that delay to the scheduler. An instrumented reuse trial retained one client transport error and confirmed delays before sending headers. A fresh-connection trial completed all 768 calls with much shorter pre-proxy delay. Connection reuse is therefore an explicit harness setting; these single-run comparisons do not establish a production-client tuning rule. These are finite single-run integration checks, not hardware capacity or policy-quality results: the fake worker has fixed service delay and no model kernels, cache behavior, transfer cost, or token streaming. Exact configurations, source hashes, compressed raw evidence, and limitations are recorded in the repository's HTTP load-baseline report.

Repeated control attribution. A follow-up uses three paired seeds with periodic control enabled or disabled, alternating run order at 256 and 1,024 sessions. Proxy counters distinguish returned predictions from caller timeout fallback, other errors, and cancellation. Control-off also removes refreshed board state and policy actuation, so this is an operational ablation rather than an equal-information policy comparison. An inherited 1,024-file-descriptor limit caused socket-accept exhaustion in some large cases; those failed trials are retained separately. Repeating the large cases with a process-local limit of 8,192 completed all 18,432 calls.

With eight fake-worker slots and 50 ms service, measured throughput was 147–151 calls/s, close to the ideal service bound of 160 calls/s. Client p95 across the three runs was 8.17–8.28 s with control and 8.27–8.69 s without. Despite similar saturated throughput, pooled prediction timeouts were 37.47% with control versus 10.88% without; maximum sampled board event-loop lag was 197–214 ms versus approximately 2 ms. Control computation, populated state, policy actions, and changed request timing remain combined in this contrast.

A separate series gives the fake worker 64 slots and 5 ms service and sets the proxy window to 64. All 18,432 calls succeed, but finite-workload throughput is only 307–345 calls/s. No worker waiters are observed; proxy process CPU is 9.08–10.17 s during approximately 8.9–10.0 s of workload, consistent with roughly one busy core, with slightly different accounting windows. Caller prediction timeouts remain 46.41% without control and 56.98% with it. Client p95 ranges overlap, and the sign of the paired latency difference varies. Generator lateness and transport costs are also material. These measurements motivate profiling the proxy request path and isolating deadline-sensitive prediction handling from control work; they do not identify a particular hot function, establish sustained serving capacity, or justify a language rewrite. The repository's repeated-control-attribution report retains configurations, source hashes, compressed raw evidence, and individual results for all 24 trials.

Proxy profiling and a transport dependency correction. Per-thread CPU profiling of the fast-worker workload identified repeated missing-module searches in HTTPcore's asynchronous-runtime detector. The tested serving environment lacked its optional `sniffio` dependency. Two instrumented runs spent 4.10–4.44 s in runtime detection on the serving thread, versus 1.32–1.33 s scanning queued peer indices and 0.58–0.61 s publishing events. These are profiler-inflated cumulative function timings, not uninstrumented request costs. Supplying `sniffio` reduced the detector to 0.067–0.068 s in follow-up profiles. A focused unprofiled test reduced 10,000 checks from a seven-repeat median of 255.11 ms to 2.45 ms.

Three paired seeds per control mode then compared otherwise identical package environments with and without the dependency, alternating trial order. All 36,864 calls succeeded. With periodic control, the median of run p95 latencies fell from 3.495 to 2.437 s and median proxy CPU from 9.046 to 6.892 s; without control, these changed from 4.035 to 3.007 s and 9.772 to 7.825 s. However, pooled prediction timeout rates increased from 55.62% to 63.31% with control and from 38.43% to 47.81% without. The dependency is available to the generator and all services, so the integrated effect includes changed request timing and prediction completion patterns. These results support removing redundant import work but do not

demonstrate better scheduling quality or establish a universal speedup. The serving extra now supplies the detector explicitly; scheduling algorithms are unchanged. Those trials motivated the subsequent bounded prediction path described below. Thread-aware profiling remains an optional experiment dependency, and the repository’s proxy-profiling report preserves all individual trials and measurement limitations.

Behavior-preserving maintenance. A subsequent core refactor consolidates sidecar execution and configuration, simulator callbacks and index calculations, and trace/forecast schema helpers. Ten deterministic simulation policy variants preserve request, session, and random-generator-state hashes; an 8,192-session forecast preserves its result hash. A shared per-line parser helper was rejected after a 4.9% microbenchmark regression, retaining inline parsing. The retained sidecar paths preserve output and event hashes; their 21-sample medians differ by +1.8% for wrapped output and -0.1% for subprocess output. These short synthetic measurements support the maintenance decision, not universal performance equivalence. The final suite passes 420 tests with five skips. The dated core-refactor report records the fixtures, intermediate revisions, raw timings, and validation limits.

Implementation structure and equivalent execution. The September 28 modularity pass separates application handlers, predictor fitting, event processing, simulation transitions, and experiment reporting into functions of at most 20 source lines. Explicit depth-first stacks replace recursive synthetic-family generation and cloning while preserving seeded draw order. The GDP implementation caches its exact hold-window boundary search by starting slot within each planning pass; feasibility itself remains uncached because committed demand changes. For N sessions, S slots, and $U \leq S$ distinct starting slots, boundary-search work is $O(U \log S + N)$ with $O(U)$ storage. This is an implementation change, not an additional forecast or policy claim. The reproducibility record in docs/research/results/modularity-2026-09-28/ reports output equivalence and CPU timing limits.

Bounded prediction admission (September 28). The proxy now admits at most L unfinished prediction jobs ($L = 4$ by default), with immediate local fallback when full. Its monotonic caller deadline includes executor dispatch and prediction; an expired job skips computation. A timeout or caller cancellation does not release capacity held by a running thread: only future completion does. Board HTTP phase timeouts use the remaining caller budget rather than a 500 ms minimum. The caller also rejects results after its deadline. Event-loop stalls can delay timeout handling, and remote computation may outlive the HTTP request; neither hard real-time behavior nor server-side cancellation is claimed.

Admission and accounting require $O(1)$ work per request and $O(L)$ unfinished jobs. Counters distinguish immediate rejection, caller timeout, prediction use, and outstanding/running/completed worker jobs. Shutdown drains the executor before closing owned clients. This preserves fallback estimates and scheduling formulas but can change which predictions are used and therefore request order. At that revision, board computation still ran synchronously in its service handlers; the following implementation isolates that work. Paired CPU-only HTTP trials and their source hashes are retained in the September 28 prediction-overload study.

In three paired fast-worker trials at each size, client p95 decreased in all six pairs: the three-run ranges changed from 0.503–0.825 to 0.427–0.557 s at 256 sessions and from 2.311–3.465 to 1.851–2.926 s at 1,024 sessions. However, pooled timely prediction use fell from 58.42% to 52.00% and from 32.26% to 18.98%, respectively; immediate rejection accounted for 36.98% and 71.69% of candidate attempts. All 23,040 planned calls completed, and candidate peak unfinished prediction work was four. This is a bounded-work result with a latency/coverage trade-off, not evidence of improved policy quality. Ranges are not confidence intervals. Baseline shutdown logs contain a pre-existing cleanup failure after measurement; the candidate lifecycle removes it. The study retains both versions’ logs, configuration and source hashes.

Board execution isolation and freshness (September 28). The next implementation gives one bounded control worker ownership of event application, registry/model state, random draws, forecast construction

and controller planning. Extra control requests receive HTTP 503; cancellation does not release a running mutation. A successful tick atomically publishes a scalar prediction view, version, monotonic capture time and pre-encoded forecast JSON. HTTP prediction reads use the last complete view while the next tick runs. A failed tick retains that publication, without claiming transactional rollback of consumed events. This avoids copying mutable session histories or the model.

For S sessions and D duration observations visited across distinct tool means, projection takes $O(S + D)$ time and $O(S)$ storage. Built-in request prediction uses expected $O(1)$ lookup and the unchanged service-time formula. Forecast and planning algorithms retain their complexity and seeded draw order. Snapshot quantiles and directive JSON are computed on the worker. The publication lock covers a reference read/swap, not numerical work. Custom overridden prediction methods retain a separate bounded legacy runner; their arbitrary state is not projected. Threads still share the Python GIL.

For monotonic capture time c , request time t and age limit A , the view is unavailable when $t - c > A$. Capture starts before ingestion, so computation counts toward age. The default is $A = \max(1 \text{ s}, 3\Delta)$ for tick interval Δ ; operators can override it. Stale results trigger existing local fallback.

Fresh paired trials compare a651518 with 2bc3f8b, with three seeds at each of 256 and 1,024 sessions. Timely prediction coverage rises from 55.82% to 63.80% and from 24.59% to 31.61%, respectively. At 1,024 sessions, board heartbeat p95 delay falls from 88.935–92.318 ms to 1.053–1.566 ms, but client p95 rises from 2.183–2.730 s to 3.100–3.190 s in all three pairs. Three additional pairs with zero imposed hold delay also regress in client p95. All 41,472 calls succeed across both experiments. This establishes improved board responsiveness and availability, not an end-to-end latency win. Different enqueue timing, prediction transaction counts and later control inputs prevent attributing the regression to one cause. The dated board-isolation study retains all raw evidence, source hashes, stage timings, test results and host limitations.

Proxy request phases and exact peer ranks (28 September 2026). The follow-up profile at 36f0860 identified upstream HTTP pool maintenance and peer-list construction as larger main-thread costs than anticipated. Completed traces now distinguish monotonic prediction waiting, enqueue-to-admit waiting (including holds), and admit-to-handler-dispatch delay. Missing phase fields in older traces are omitted. Wall-clock differences can be invalidated by clock adjustments; separate phase percentiles are not additive.

The retained implementation at 3084f34 maintains exact per-tier counts of queued index values. With n queued peers and b indices strictly below the outgoing request, the hint is $\min(3, \lfloor 4b/n \rfloor)$, or 3 for an empty tier. Held peers count and released peers do not. Submission, cancellation, release and promotion update counts in expected constant time. A query scans U_i distinct values in its tier instead of materializing a full peer list and NumPy array; additional storage is $O(U)$ across tiers. Rebuilds remain $O(Q)$. When priorities are all unique, U_i may equal backlog Q , so ranking an entire drain can still cost $O(Q^2)$. Scheduling heaps and hold semantics are unchanged.

A query microbenchmark with 100,000 peers measured 6.21 ms for the old list/NumPy path versus 2.10 ms for the histogram when all indices differ, and 6.24 ms versus 0.72 μ s with 16 distinct values. These are medians of seven repetitions, excluding construction and mutation. In a separate instrumented 1,024-session run, all 3,072 requests completed; histogram queries used 0.029 CPU seconds, compared with 1.278 seconds for peer-list collection alone in the discovery profile. Instrumentation changes scheduling and is not latency evidence.

Two separate paired studies used three seeds at 256 and 1,024 sessions, 64-slot proxy and fake worker, 5 ms service, and identical phase instrumentation at baseline f180401. All 46,080 unprofiled requests succeeded. The retained rank change reduced proxy CPU in six of six pairs; client p95 improved in four and regressed in two. At 1,024 sessions the per-run p95 range changed from 2.564–4.067 s to 1.823–2.630 s, but baseline variability prevents a robust single speedup estimate. The seed-7 pair regressed from 2.564 to 2.630 s. These short localhost trials use no GPU, KV transfers or token streaming.

The other candidate increased upstream idle connection retention from 20 to 64. Despite passing real-HTTP reuse tests, its large-case client p95 increased from 2.681–2.743 s to 4.087–4.466 s. A diagnostic

profile opened 70 rather than 3,072 upstream connections, but HTTPcore pool-assignment CPU increased from 2.674 to 11.836 s. That profile had one failed request and two unissued turns; it is retained as incomplete diagnostic evidence. The pool change was reverted in 880b232; transport defaults remain unchanged. The retained rank profile still spent 2.550 CPU seconds in pool assignment. The dated 2026-09-28-proxy-ranking.md study archives all trials, profiles, source hashes, phase results and tests, including unfavorable results. The full retained suite passes 496 tests with three skips; Python functions remain at most 20 lines.

Bounded upstream pool sharding (28 September 2026). A follow-up transport-only fixture separates HTTP work from forecasting and admission. The retained implementation at 0425a58 balances requests across $P = 16$ independent HTTPX pools sharing a total connection budget $C = 100$. Pool i receives $\lfloor C/P \rfloor$ slots plus one when $i < C \bmod P$. Selection minimizes unfinished response count divided by pool capacity, scanning $O(P)$ counters. Counts include waits for headers and open streaming bodies; pre-header failures or response close return occupancy exactly once. One shared TLS context retains certificate and hostname verification. Standard HTTP proxy discovery retains stock HTTPX routing. The wrapper uses public transport APIs, without patching HTTPcore, changing admission priority or adding retries.

The aggregate idle cap increases from 20 to 100 with the same five-second expiry; no connections are opened eagerly. Each smaller pool still performs internal scans and assignment retries. With fixed P and C , this is a constant-factor optimization, not a new fleet-size complexity bound. Requests already assigned to one pool are not migrated when another pool becomes available. Imbalanced stream lifetimes and overload can therefore leave local queueing. Setting `upstream_pool_shards=1` restores the stock transport.

The focused fixture uses only a load client and fake worker with 5 ms service, three reversed-order repetitions and fresh clients. At 64 concurrent requests and 3,072 calls per variant, stock client CPU was 1.979–2.052 s with per-run p95 56.52–59.29 ms; 16 shards used 1.278–1.305 s with p95 40.19–45.70 ms. Client construction and close are outside timing; TCP setup and trace recording are included. At eight concurrent calls, sharding reduced CPU but increased p95 from 7.66–8.32 to 8.54–9.08 ms. Thus 321bf59 adds automatic stock selection for initial admission windows of 20 or fewer; larger windows select 16 shards unless explicitly overridden. This threshold follows the stock idle cap and is not a universally calibrated crossover. A lightly used large window may still incur the small-load cost. Across 8, 64 and 256 concurrency, all 82,944 formal component requests succeeded, including demand above the connection cap.

Six fresh full-proxy pairs compare bb2a561 with 0425a58, using three seeds at 256 and 1,024 sessions and the previous 64-slot burst workload. All 23,040 requests succeeded. CPU and client p95 improved in every pair. At 1,024 sessions, p95 changed from 2.462–2.654 s to 1.422–1.465 s, and proxy CPU from 6.840–7.099 s to 4.459–4.750 s. Timely prediction use fell from 35.50% to 31.93%; caller timeouts rose from 78 to 1,485 across the three large runs, while immediate rejections fell. Prediction-wait p95 increased to 0.108–0.136 s despite the unchanged 50 ms caller budget, exposing event-loop resumption delay rather than a hard deadline guarantee. The fixture's $\beta = 0$ and identical service/fallback rates do not establish the policy cost of lower prediction coverage. That tradeoff requires prediction-sensitive trials.

A separate candidate CPU profile completed 3,072 calls, opened 64 upstream TCP connections and spent 0.798 CPU seconds in HTTPcore connection assignment. Instrumented timings are diagnostic, not paired latency evidence. The final small-window policy retains the measured 16-shard selection for the 64-slot workload; these numbers remain attributed to the measured revision. All 528 tests pass with three skips, including real-socket partial-stream cleanup, connection reuse, cancellation, close failures, proxy fallback and capacity boundaries. The dated 2026-09-28-sharded-transport.md study preserves all configurations, hashes, raw evidence and limitations. Remote TLS/streaming performance, GPU benefits and production capacity remain unvalidated.

5 Experimental Methodology

The evaluation separates three links in the argument. H1a tests fleet forecasts using held-out session traces; H1b tests remaining-time predictions using instrumented tools; H2 tests the consequences of changing admission in a closed loop. These experiments answer different questions: a low forecasting loss does not by itself establish a beneficial serving policy.

5.1 Data and splits

Table 4: Data sources and evaluation units. Corpus counts describe the repository’s recorded input snapshot rather than a guarantee about current upstream releases.

Source	Recorded size	Evaluation protocol
TraceLab [7]	4,265 sessions; approximately 357,000 model steps	H1a: 30% of calendar-week blocks held out; test sessions overlaid as a Poisson fleet.
AgentX [8]	393 root sessions; 1,697 sub-agent groups	H1a: 30% of session families held out; roots and children split and shifted together.
Instrumented tools	202 observation points per model across nine scored job families	H1b: one job family held out at a time; real builds, pipelines, and NumPy tests.
Synthetic programs	Five base configurations plus two rule revisions; three seeds each	H2: one hour of new-session arrivals, followed by completion; matched programs across policy arms.

H1a uses arrival rates of 100, 200, and 400 sessions per hour, four-hour overlay windows, 30-second forecast ticks, 256 Monte Carlo samples, and horizons of 10, 30, 120, 300, and 900 seconds. The seed controls splitting and overlay generation. Both public corpora are assigned to the interactive class by their adapters, so these measurements are not evidence for forecast accuracy across independently labeled interactive and background populations.

TraceLab’s adapter aggregates tools launched by a model step into one phase and uses the longest tool to label that phase. AgentX’s inter-request gaps are not individually labeled as tool or human time; they are represented as generic gaps unless associated with a sub-agent group. Keeping families together prevents related root and child traces from crossing the train/test boundary, but Poisson overlays do not preserve production arrival correlations.

H1b scores observations every 15 seconds within tool phases longer than 30 seconds. A job family is defined by the collection job name, so the same tool at other sizes or rates may remain in training. The retained score file contains nine families: two fmt build variants, four linear pipelines, two staged pipelines, and the NumPy suite. All observations in a held-out family are scored using models trained on the remaining families. Results are averaged over observation points; long phases therefore receive more weight, and the 202 points are not independent experimental replicates.

5.2 Forecast metrics

The primary forecast metric is pinball loss at quantile $\tau = 0.9$, with supplementary continuous ranked probability score (CRPS) and interval coverage [9]. For observation y and predicted quantile q_τ ,

$$\ell_\tau(y, q_\tau) = (\tau - \mathbf{1}[y < q_\tau])(y - q_\tau). \quad (22)$$

An equal-magnitude underprediction receives nine times the penalty of an overprediction at this quantile. Scores retain the target’s units: KV blocks for H1a and seconds for H1b. A loss ratio describes improvement in this score, not a speedup or a corresponding reduction in mean absolute error. H1a sweep ratios divide the mean B1 loss by the mean M1 loss across seeds, rather than averaging per-seed ratios.

CRPS evaluates the full predictive distribution F , rather than one quantile:

$$\text{CRPS}(F, y) = \int_{-\infty}^{\infty} (F(z) - \mathbf{1}[y \leq z])^2 dz. \quad (23)$$

Here z ranges over possible target values, and $F(z)$ is their predicted cumulative probability. Lower CRPS is better. For N evaluated forecasts, empirical central 90% coverage is

$$\widehat{C}_{90} = \frac{1}{N} \sum_{k=1}^N \mathbf{1}[q_{0.05,k} \leq y_k \leq q_{0.95,k}], \quad (24)$$

where k indexes forecast–observation pairs. Coverage measures interval reliability; wide intervals can cover well while remaining uninformative, so it is read alongside loss.

5.3 Monte Carlo error and the meaning of a quantile decision

A finite sample introduces numerical uncertainty even if the predictive model were correct. Conditional on that model and independently sampled scenarios, Equation (18) has

$$M\widehat{p}_K \sim \text{Binomial}(M, p_K), \quad \text{SE}(\widehat{p}_K) = \sqrt{\frac{p_K(1-p_K)}{M}}, \quad (25)$$

where p_K is the model’s exact exceedance probability. At $p_K = 0.1$, this standard error is approximately 2.65 percentage points for 128 draws and 1.88 for 256 draws. These are sampling errors around the model’s probability, not uncertainty about whether the model describes reality. The required sample count for a chosen standard-error target δ is at least $p_K(1-p_K)/\delta^2$; the worst-case bound uses $p_K(1-p_K) \leq 1/4$. Increasing sample count cannot correct biased or miscalibrated predictions.

Quantile decisions also encode asymmetric penalties. For a continuous predictive CDF F , differentiating expected pinball loss with respect to the forecast q gives

$$\frac{d}{dq} \mathbb{E}[\ell_\tau(Y, q)] = F(q) - \tau. \quad (26)$$

Thus a τ -quantile minimizes that expected loss; for discrete distributions the corresponding statement uses subgradients. More generally, minimizing expected underage/overage cost

$$\mathbb{E}[c_u(Y - q)_+ + c_o(q - Y)_+] \quad \text{selects} \quad \tau = \frac{c_u}{c_u + c_o}, \quad (27)$$

where $c_u, c_o > 0$ are costs per unit of underestimated and overestimated demand. Selecting q90 corresponds to a 9:1 ratio in this one-dimensional loss. It does not demonstrate that the serving system’s true costs have this ratio, or that a 90th-percentile demand threshold achieves 90% session SLO attainment. The two metrics concern different random variables and different decisions.

Dispersion calibration is another operation on the distribution, separate from Monte Carlo precision. In the implementation, samples at a fixed class and horizon are transformed as

$$\widetilde{D}^{(m)} = \max\{0, \bar{D} + \kappa(D^{(m)} - \bar{D})\}, \quad \bar{D} = \frac{1}{M} \sum_m D^{(m)}, \quad (28)$$

where $\kappa \geq 1$ is selected on held-out training observations to increase central-interval coverage. Without clipping, the sample mean is preserved and variance is multiplied by κ^2 . Clipping negative transformed draws at zero can increase the mean, so the implemented transformation is not strictly mean-preserving. Coverage, bias, and upper-quantile loss must therefore all be checked after calibration; there is no distribution-free coverage guarantee under the present fitting procedure.

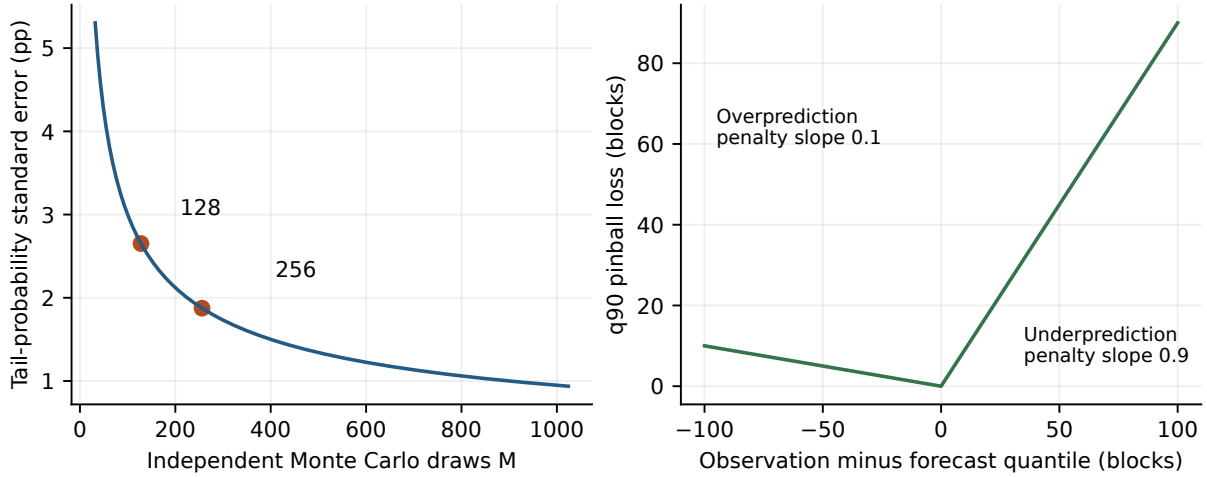


Figure 12: Two distinct consequences of using a probabilistic forecast. Left: analytic Monte Carlo standard error for a model tail probability of 0.1, with the study’s 128- and 256-draw settings marked. Right: q90 pinball loss versus signed prediction error, showing its asymmetric slopes. Neither panel reports empirical accuracy or a serving guarantee.

5.4 Closed-loop workloads and baselines

The simulator advances each program’s next turn after completion of its previous inference and tool execution. It models a block-based least-recently-used cache, prefix reuse, a batch limit, priority order, and fixed prefill/decode rates of 20,000 and 40 tokens per second, respectively. Training programs use an independent seed offset of 1,000 and are run under the native policy. Forecast policies use 128 samples and horizons of 30, 120, and 300 seconds.

Table 5: Completed simulator configurations. Rates are new sessions per hour; KV capacity is in 16-token blocks. Long/short refers to background tools unless stated otherwise.

Regime	Interactive / background rate	KV / batch / window	Hold cap (s)
Long tools	120 / 240	8,000 / 8 / 8	600
Short tools	120 / 240	8,000 / 8 / 8	600
Loaded long tools	360 / 600	6,000 / 6 / 6	600
Long interactive tools	300 / 480	6,000 / 6 / 6	60
Loaded long tools, cap ablation	360 / 600	6,000 / 6 / 6	60

Background programs have a mean of ten turns and a 1,800-second deadline. In the long-tool regimes, tool durations are lognormal, with medians of 3 seconds for shell commands, 180 seconds for test suites, and 400 seconds for builds. Their selection probabilities are 0.5, 0.3, and 0.2. Test suites expose strong progress signals and have a 0.05 spawn probability; builds expose weak signals. The short-tool regime uses shell commands and tests with medians of 2 and 20 seconds. Interactive programs have a mean of six turns and a lognormal think-time distribution with median 20 seconds. The long-interactive regime adds 120-second tests and 300-second builds to that class.

Seven arms are evaluated in the first three regimes: *native* engine priority, *rules* with a proxy window and index, *forecast M1*, *forecast M2*, *M2 without holds*, *heap lookahead*, and a *working-set* policy. The working-set policy pauses background programs and evicts their cached state when occupancy exceeds a budget, resuming with hysteresis. The long-interactive experiment adds *same-rule lookahead*, which feeds demand inferred from the current event heap into GDP-lite. Both lookahead arms also use actual next-tool durations in the index. They are diagnostic comparisons, not optimal oracles: the heap does not enumerate all future events that policy decisions may induce, and the information and index differences prevent a clean attribution solely to forecast error.

5.5 Serving metrics and uncertainty

Time to first token (TTFT) is measured from request arrival for interactive calls after the initial turn. For each interactive session, the evaluator computes the fraction of these calls with TTFT at most two seconds; *session-weighted SLO attainment* is the mean of these fractions. It is not the fraction of sessions for which every call meets the target. Background job completion time (JCT) is measured from session start to end. Deadline attainment and recomputed prompt tokens quantify additional costs.

In notation, let $\mathcal{J}$ be the evaluated interactive sessions, n_j the number of eligible post-initial calls in session j , and ℓ_{jk} their TTFT values. Then

$$\text{SLO} = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} \frac{1}{n_j} \sum_{k=1}^{n_j} \mathbf{1}[\ell_{jk} \leq 2 \text{ s}], \quad \overline{\text{JCT}} = \frac{1}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} (f_j - a_j), \quad (29)$$

where $\mathcal{B}$ is the evaluated background-session set and a_j, f_j are session start and finish times. Sessions with no eligible post-initial call are excluded from $\mathcal{J}$. This weighting gives each included interactive session equal influence, even when sessions have different numbers of calls. An SLO difference of 0.01 corresponds to one percentage point.

Arms share generated programs and seeds. The reported 95% intervals use 300 paired bootstrap draws over common session identifiers pooled across three seeds. Difference estimates are means of the bootstrap differences; they may differ slightly from subtraction of the displayed point estimates. These intervals reflect variability among sampled sessions under the selected workloads. They do not account fully for dependence between sessions sharing a queue, uncertainty across workload families, or simulator error. No claim of statistical equivalence follows from an interval containing zero.

6 Results

We first ask whether the observable state improves the forecast, then whether progress adds information, and finally whether a policy can use that information profitably. The subsequent comparisons stress the policy by changing the workload, changing only the hold cap, and revising the capacity estimate.

6.1 H1a: in-flight state improves medium-horizon forecasts

Table 6 presents the original TraceLab run at 200 sessions per hour. M1’s q90 pinball loss is lower than B1’s at horizons from 30 seconds through 15 minutes, while B1 performs better at 10 seconds. At five minutes, the B1/M1 ratio is 2.15; at fifteen minutes it is 3.39. M2’s small numerical differences from M1 are attributable to sampling because the corpus supplies no live progress observations.

Table 6: TraceLab, seed 0, 200 sessions/hour. Mean q90 pinball loss of first-resumption KV demand, in blocks; lower is better. Source: the original H1 run, not the three-seed aggregate.

Model	10 s	30 s	2 min	5 min	15 min
B0 persistence	10,409	10,845	13,861	16,485	43,105
B1 Kalman	4,588	6,615	10,597	14,186	26,101
B2 history	40,651	38,324	33,861	30,628	26,183
M1 survival	5,567	5,254	5,679	6,604	7,700
M2 progress	5,538	5,251	5,654	6,611	7,671

Across the sweep, ratios at five minutes range from 1.96 to 3.31 on TraceLab and from 2.97 to 5.43 on AgentX (Figure 13). At fifteen minutes, the ranges are 3.00-5.09 and 2.73-3.58, respectively. AgentX’s advantage therefore does not increase monotonically with horizon. Three seeds provide limited precision:

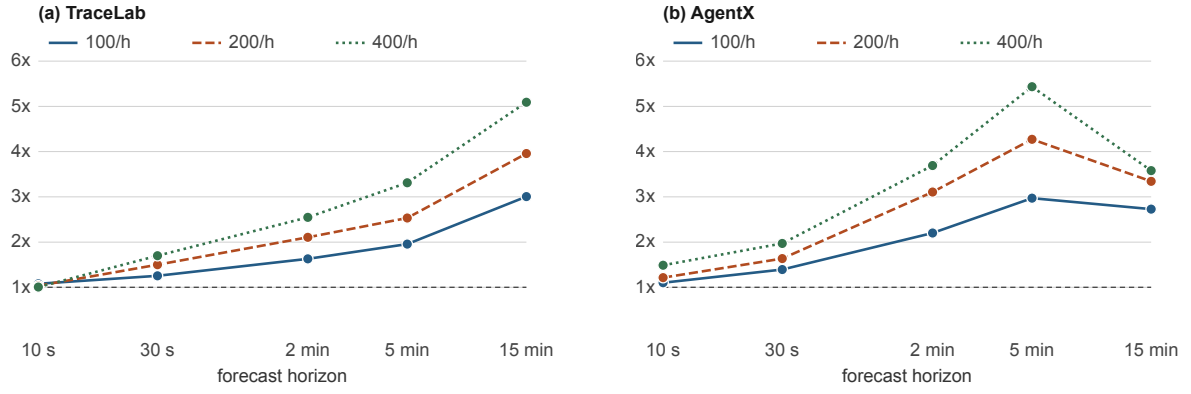


Figure 13: H1a sweep: ratio of mean B1 loss to mean M1 loss across seeds 0-2, at three overlay arrival rates. Ratios above one favor M1. The horizontal axis is logarithmic; line styles also distinguish rates. These are point estimates, with no confidence interval for the ratio. The near-unity TraceLab values at 10 seconds do not imply consistent improvement on individual seeds.

the stored normal-approximation interval half-widths for M1’s fifteen-minute AgentX loss are approximately 51-63% of its mean. These intervals describe the component loss, not the plotted ratio.

The recorded TraceLab analysis attributes 81-99% of demand to sessions already in flight over the tested horizons, consistent with the opportunity for session-aware forecasting. However, nominal 90% intervals from the session model cover only approximately 40-73% of observations. Training-only dispersion calibration increases coverage to approximately 65-86% and worsens short-horizon pinball loss in the recorded calibration run. Residual bias, dependence, and distribution shift are plausible explanations; these results do not identify which mechanism dominates. B2 performs poorly in the heterogeneous fleet experiments, supporting survival conditioning for this predictor rather than establishing the inferiority of every TTL-based cache policy.

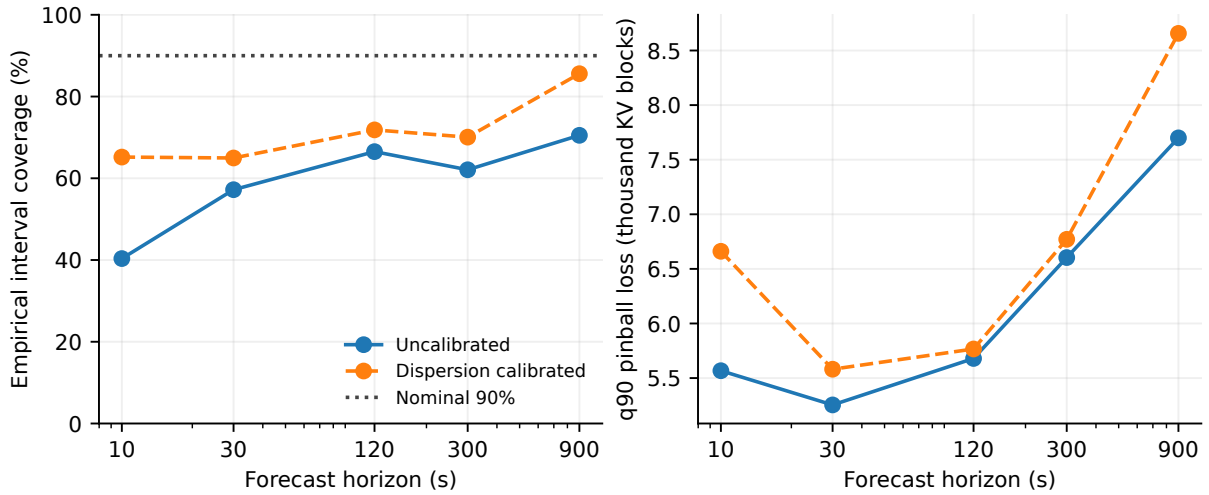


Figure 14: Accuracy and calibration must be checked separately. TraceLab at 200 sessions/hour: M1 before and after training-only dispersion calibration, using the retained original and calibrated runs. Left: central 90% interval coverage versus its nominal target. Right: q90 pinball loss. These are descriptive run-level summaries, without cross-seed uncertainty estimates.

Figure 14 shows why the loss gains do not establish a calibrated probability model. Increasing dispersion moves coverage toward its nominal level, but can worsen the decision-relevant upper-quantile score. The next experiment isolates a different information source: observations of the work a tool has actually completed.

The model hierarchy is also visible in Figure 15, which normalizes the original run by its Kalman loss

separately at each horizon. This view separates a short-horizon failure from the medium-horizon gains: a model can be useful over part of the horizon range without dominating everywhere. It also shows why conditioning on survival and adding progress are distinct comparisons; the corpus has elapsed-time state but no live progress feed.

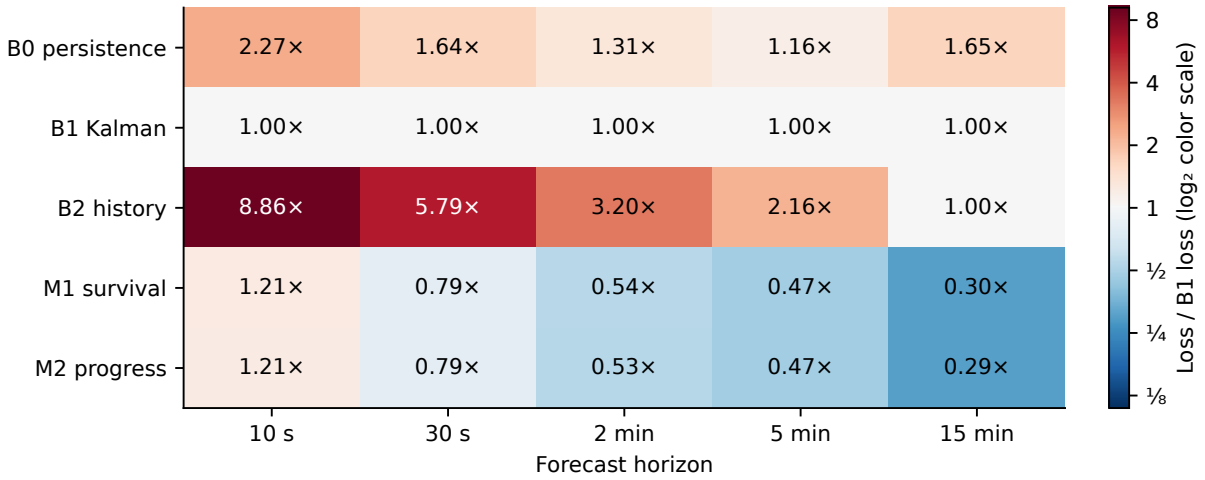


Figure 15: Empirical q90-loss ratios in the original TraceLab seed-0 run at 200 sessions/hour. Every cell divides the model loss by B1 loss at the same horizon; values below one favor the row model. Color uses a base-2 logarithmic scale centered at one, while labels show the untransformed ratios. This is a descriptive single-seed comparison, not the three-seed sweep or a significance test.

6.2 H1b: progress helps, but depends on the tool’s progress curve

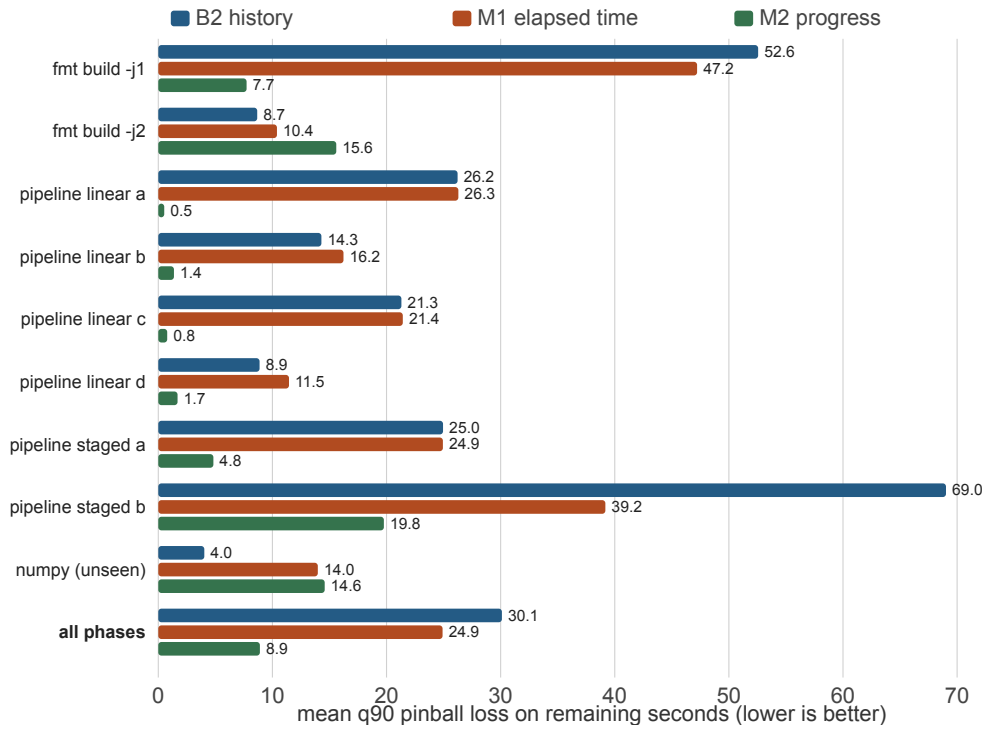


Figure 16: Leave-one-job-family-out remaining-time evaluation. Bars show mean q90 pinball loss in seconds; lower is better. The aggregate weights all 202 observation points equally. The NumPy family is unseen in training for its displayed row. Bars are descriptive averages, not estimates from independent observations with uncertainty bars.

Across the retained collection, M2 reduces mean q90 pinball loss from M1’s 24.9 to 8.9 seconds, a

2.80-fold loss ratio (64% reduction). CRPS decreases from 59.0 to 30.1 seconds, a 1.96-fold ratio. The larger CRPS ratio of 3.4 recorded for an earlier pipeline/build-only collection does not apply to the expanded collection shown here.

Progress is most informative for the linear pipelines, whose M2 losses range from 0.5 to 1.7 seconds. It remains useful with four stage markers, but is not uniformly beneficial: M2 loses to M1 for the `fmt - j2` build and the held-out NumPy suite. Reported work fractions can map nonlinearly to elapsed time, and a curve learned from a different build configuration can be misleading. A separate one-prior-run NumPy analysis in the research notes reports CRPS of 18.8 seconds for M2 versus 53.7 for M1; that is a different training protocol and is not pooled into Figure 16.

The results support the value of progress on the measured jobs, not on arbitrary external actions. The sample contains nine scored families on one machine, with repeated observations within phases and limited environmental variation. The current implementation can key progress curves by command signature, but the retained score file predates evaluation of that change. Its results must not be attributed to the newer signature-aware implementation.

6.3 H2: loaded short-interactive workloads show a latency-cost trade-off

We now turn from prediction to action. Figure 17 places the five base configurations on common outcome axes. Large gains occur in the loaded 600-second-cap configuration; the remaining configurations expose either little headroom for improvement or an unfavorable background cost. The following subsections examine these cases rather than averaging unlike workloads together.

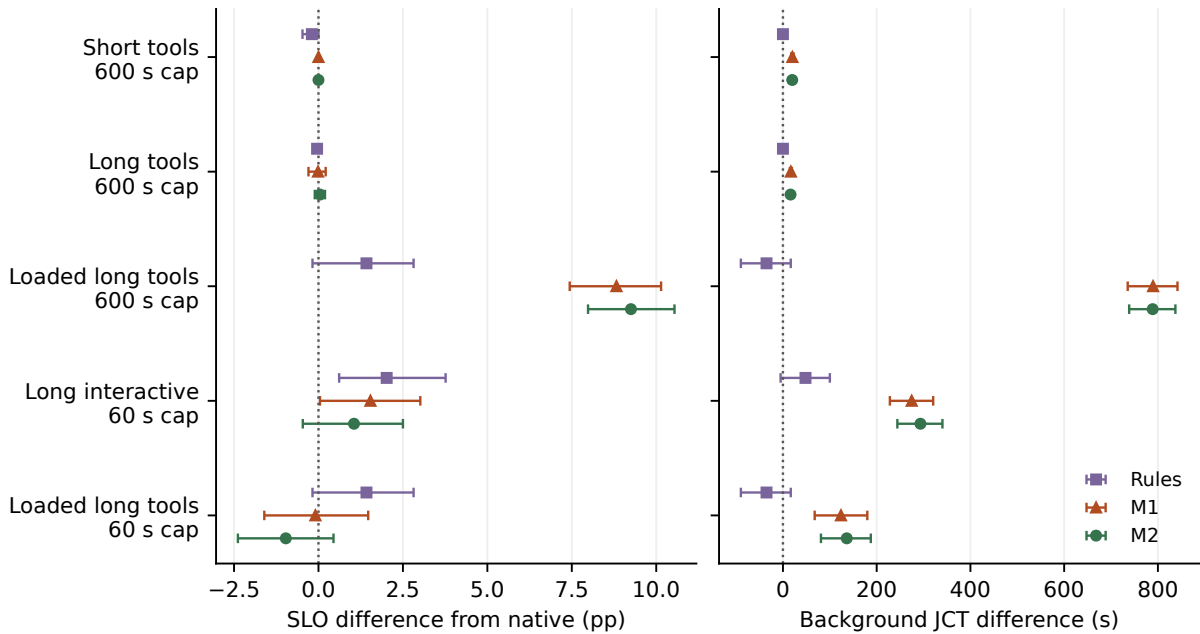


Figure 17: Policy effects across the five base configurations. Points show paired differences from native; lines are 95% session-bootstrap intervals. Positive SLO differences are benefits; positive background JCT differences are costs. Rows denote separate workloads except the two loaded-long-tool rows, which share the workload and vary the cap. Intervals pool sessions across three seeds and do not measure uncertainty across workload families.

In the loaded long-tool regime, interactive sessions still execute short shell tools. Forecast M2 raises session-weighted SLO attainment from 85.3% to 94.6% (Table 7). Its paired improvement is 9.3 percentage points, with a 95% session-bootstrap interval of 8.0-10.5 points. Mean background JCT rises from 3,310 to 4,100 seconds, approximately 24%. The corresponding bootstrap estimate of the difference is 789 seconds [739, 837].

Removing holds from M2 eliminates the observed SLO gain over native in this regime. This supports

Table 7: Loaded long-tool regime, three seeds. SLO is session-weighted attainment (%); Δ SLO is in percentage points. JCT is mean background completion time (s). Brackets are paired 95% session-bootstrap intervals.

Arm	SLO (%)	Δ SLO [95% interval]	JCT (s)	Δ JCT [95% interval]
Native	85.3	-	3,310	-
Rules	86.7	+1.4 [-0.2, +2.8]	3,278	-35 [-90, +17]
Forecast M1	94.1	+8.8 [+7.4, +10.1]	4,101	+790 [+736, +842]
Forecast M2	94.6	+9.3 [+8.0, +10.5]	4,100	+789 [+739, +837]
M2, no holds	84.1	-1.3 [-2.8, +0.1]	3,277	-35 [-97, +10]
Heap lookahead	92.3	+7.0 [+5.4, +8.3]	4,453	+1141 [+1090, +1190]
Working set	92.9	+7.6 [+6.3, +8.9]	5,392	+2082 [+2035, +2136]

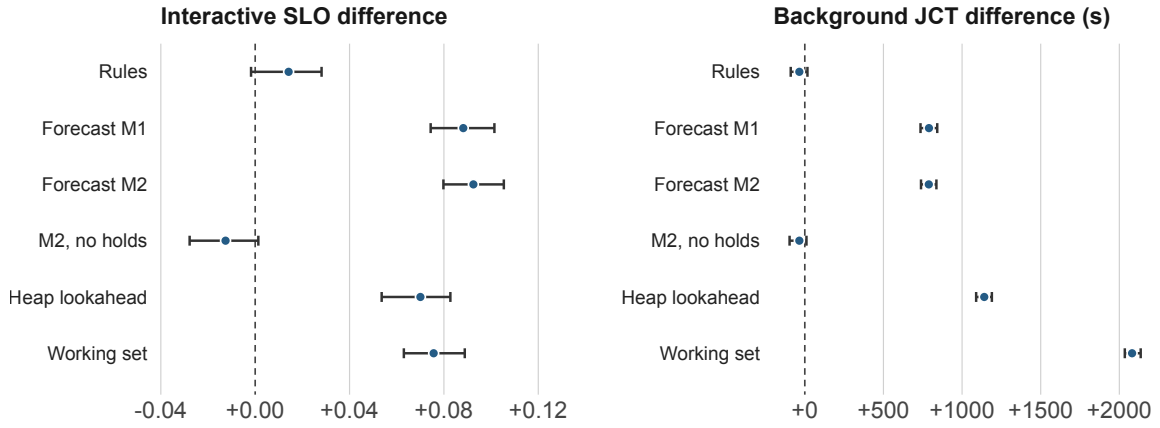


Figure 18: Paired differences from native in the loaded regime. Left: session-weighted SLO difference on the 0-1 scale. Right: mean background JCT difference in seconds. Points are bootstrap mean differences; lines are 95% session-bootstrap intervals. Positive SLO differences are beneficial; positive JCT differences are costs.

a role for the hold mechanism in the measured trade-off; it does not prove that the particular forecast is necessary or that another hold heuristic could not obtain a similar result. M1 and M2 produce similar point estimates because the hold rule targets interactive demand over 30 seconds, while the distinctive long progress-emitting tools are in the background class.

The working-set arm reaches 92.9% SLO attainment and adds 2,082 seconds of background JCT. Its incremental background cost is approximately 2.64 times that of M2, but the SLO outcomes are not identical. This is a comparison between local policies under one simulator configuration, not a claim of superiority over ThunderAgent. Neither lookahead nor working-set results provide an optimal policy bound.

Costs extend beyond JCT. Forecast M2 reduces background deadline attainment from 19.0% to 12.2%, accumulates approximately 4.20 million held KV block-seconds per seed, and reaches the hold cap on approximately 1,193 occasions per seed (Appendix B). Its recomputed prefill volume is lower than native’s in this loaded regime. In the underloaded long- and short-tool regimes, near-saturated SLO attainment limits discrimination: forecast M2 adds about 16 and 20 seconds to background JCT, respectively, for little change in the SLO metric. Lower request latency is not sufficient evidence of a favorable overall policy trade-off.

6.4 Long interactive tools do not establish an M2 policy advantage

The completed long-interactive-tool experiment adds progress-emitting tests and builds to interactive sessions and uses a 60-second cap. It also changes arrival rates to 300 interactive and 480 background sessions per hour. It is therefore a separate regime, not a controlled hold-cap ablation. Table 8 includes all eight arms, including the same-rule lookahead comparison.

Table 8: Long-interactive-tool regime, three seeds, 60-second hold cap. Units and interval construction match Table 7.

Arm	SLO (%)	Δ SLO [95% interval]	JCT (s)	Δ JCT [95% interval]
Native	86.7	-	2,309	-
Rules	88.7	+2.0 [+0.6, +3.8]	2,359	+48 [-5, +100]
Forecast M1	88.2	+1.5 [+0.0, +3.0]	2,585	+275 [+228, +321]
Forecast M2	87.7	+1.1 [-0.5, +2.5]	2,604	+293 [+244, +340]
M2, no holds	86.8	+0.1 [-1.4, +1.7]	2,370	+60 [+16, +115]
Heap lookahead	87.4	+0.8 [-0.7, +2.2]	3,274	+962 [+899, +1031]
Same-rule lookahead	87.7	+1.0 [-0.4, +2.6]	3,315	+1003 [+941, +1075]
Working set	89.3	+2.6 [+1.0, +3.9]	2,921	+611 [+557, +676]

M2’s SLO improvement over native is 1.1 percentage points, with an interval spanning zero (-0.5 to 2.5 points), while its background JCT cost is 293 seconds [244, 340]. Rules-only admission has a higher SLO point estimate, 88.7% versus 87.7%, and a smaller JCT increase of 48 seconds [-5, 100]. M1 also has a higher SLO point estimate than M2. These native-referenced intervals do not directly test M1 against M2, but the results provide no evidence that progress-based forecasting improves this controller’s performance.

The same-rule lookahead arm reaches 87.7% SLO attainment with approximately 1,003 seconds of additional background JCT. More future information is thus not sufficient for this heuristic to improve the measured trade-off. Policy choice, horizon, and capacity approximation remain material limitations.

6.5 A controlled cap ablation removes the loaded-regime gain

The 60-second-cap run repeats the loaded long-tool regime of Table 7 with the same programs, seeds, engine and rates, changing only the hold cap from 600 to 60 seconds and adding the same-rule lookahead arm. It is the fixed-workload hold-cap ablation that Section 6.4 could not provide.

Figure 19 connects the two cap settings for each policy. The horizontal segments show the joint pattern: reducing deferral lowers background completion time while removing much of the interactive benefit. The plotted points are absolute outcomes; the paired intervals for differences from native remain in Tables 7 and 9.

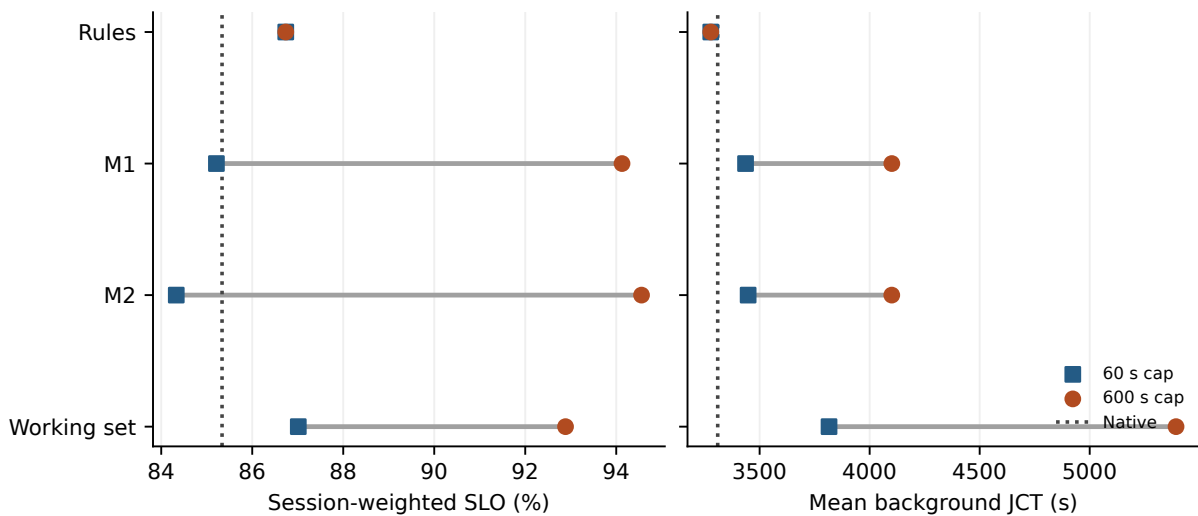


Figure 19: Controlled cap ablation on the loaded long-tool workload. Squares denote a 60 s cap and circles a 600 s cap; connecting segments identify the same policy. The dotted line is native performance, which is unaffected by the cap. Points are retained pooled-session means; no uncertainty for the direct between-cap difference is inferred from this plot.

Table 9: Loaded long-tool regime at a 60-second hold cap, three seeds. Units and interval construction match Table 7; the native row is the same workload as Table 7 and differs only by bootstrap resampling.

Arm	SLO (%)	Δ SLO [95% interval]	JCT (s)	Δ JCT [95% interval]
Native	85.3	-	3,310	-
Rules	86.7	+1.4 [-0.2, +2.8]	3,278	-35 [-90, +17]
Forecast M1	85.2	-0.1 [-1.6, +1.5]	3,436	+124 [+68, +180]
Forecast M2	84.3	-1.0 [-2.4, +0.4]	3,448	+136 [+81, +187]
M2, no holds	84.1	-1.3 [-2.8, +0.1]	3,277	-35 [-97, +10]
Heap lookahead	85.2	-0.1 [-1.5, +1.3]	4,461	+1150 [+1085, +1213]
Same-rule lookahead	84.2	-1.1 [-2.4, +0.4]	4,474	+1162 [+1099, +1226]
Working set	87.0	+1.7 [+0.2, +3.2]	3,815	+505 [+445, +566]

With the cap at 60 seconds the forecast arms’ SLO differences from native are -0.1 and -1.0 percentage points with intervals spanning zero, while their background JCT still rises by 124 seconds [68, 180] and 136 seconds [81, 187], and they accumulate approximately 3.7 million held KV block-seconds per seed with about 2,770 holds reaching the cap. The same-rule lookahead, holding with the true first-resumption demand in hand, changes SLO by -1.1 points [-2.4, +0.4] at a cost of 1,162 seconds [1,099, 1,226]. Rules-only admission is unchanged from Table 7. The working-set arm also responds to the shorter cap: its SLO attainment falls from 92.9% to 87.0%, while mean background JCT falls from 5,392 to 3,815 seconds. The 9.3-point gain of Table 7 therefore depends on allowing much longer deferrals in this experiment. The ablation establishes sensitivity to the cap; without a matched reactive-hold baseline, it does not isolate how much of that gain requires the forecast’s content.

The hold counts also suggest a mechanism consistent with Equation (21): under sustained load, little capacity is free at the instant of a decision, so many positive-demand forecasts can trigger the same hold. Similar M1 and M2 outcomes are consistent with this saturation of the decision rule; the aggregate counts alone do not prove that every decision was forecast-insensitive. A revised rule that counts capacity expected to free within the slot has also been evaluated; Section 6.6 reports the completed reruns. At the intended 60-second cap, neither loaded workload establishes a benefit for M2 over native. These findings motivate revising the capacity comparison and testing other uses of the forecast, including KV placement, rather than concluding that predictive admission cannot work.

6.6 Revised capacity accounting reduces cost without establishing an SLO gain

GDP-lite v2 augments the instantaneous free-capacity terms in Equation (21) with the blocks and slots held by running requests estimated to finish within the forecast slot. Forecast arms estimate completion from start time plus expected service time; the same-rule lookahead uses actual scheduled completion. The same loaded and long-interactive programs were rerun at a 60-second cap. This change improves the alignment of the capacity horizon, although it still does not represent a full occupancy trajectory.

In the loaded workload, M1 and M2 now add only about 3 and 7 seconds of background JCT, with intervals spanning zero. In the long-interactive workload, their costs fall to 73 and 83 seconds, respectively. SLO intervals still span zero in both workloads (Table 10). The revision therefore reduces the cost of the original rule without establishing a forecast-policy benefit over native; these intervals do not establish equivalence.

The lookahead result points to a separate attribution problem. It reaches the cap only about 10 times per seed in the loaded workload and 95 times in the long-interactive workload, yet adds 1,111 and 878 seconds of background JCT. Its use of actual next-tool duration in the ordering index is a candidate explanation. The aggregate results motivate the no-index diagnostic in Section 8; they do not by themselves quantify the causal contribution of that index. This finding reinforces the need to vary forecast information, hold logic, and priority ordering independently.

Table 10: GDP-lite v2 at a 60-second hold cap, three seeds per workload. Paired differences from native and 95% session-bootstrap intervals. SLO differences are percentage points; JCT differences are seconds.

Arm	Δ SLO [95% interval]	Δ background JCT [95% interval]
<i>Loaded long tools</i>		
Forecast M1	-1.3 [-2.9, +0.2]	+3 [-52, +49]
Forecast M2	-0.2 [-1.7, +1.1]	+7 [-52, +62]
Same-rule lookahead	-1.8 [-3.5, -0.3]	+1111 [+1042, +1178]
<i>Long interactive tools</i>		
Forecast M1	+0.1 [-1.5, +1.5]	+73 [+28, +119]
Forecast M2	+0.8 [-0.8, +2.3]	+83 [+41, +133]
Same-rule lookahead	-0.4 [-2.0, +1.5]	+878 [+822, +948]

7 Discussion and Limitations

Taken together, the figures identify two different boundaries. Session state and progress can reduce predictive loss, but imperfect calibration limits probabilistic interpretation. Admission then adds a second boundary: a forecast must be compared with a capacity model that matches its target and horizon. The limitations below follow these two boundaries from measurement through deployment.

Simulation validity. No reported policy experiment runs on a real GPU. The engine model omits important effects of actual kernels, batching, memory movement, and interconnects. Fixed service rates and simplified cache eviction can affect even the relative ordering of policies. The simulator’s metadata exposes the programmed output length to service-time estimation, which further simplifies deployment uncertainty. Mock integration establishes request-path compatibility only.

Target-to-control mismatch. First-resumption KV demand is not simultaneous memory occupancy, and input-token demand is not uncached prefill work. Comparing these quantities with available capacity is a useful heuristic to study, but forecast calibration alone cannot validate that comparison. Repeated turns, cache sharing, backend interference, and multi-level fan-out require richer models. The present forecaster models only one level of fan-out.

External validity and uncertainty. Poisson-overlaid traces preserve within-session timing while changing fleet-level arrivals. Both public corpora represent coding agents, and AgentX gaps mix tool and human delays. Tool collections favor observable, parseable progress and do not cover silent tools, network services, or human actions broadly. Three seeds, dependent sessions in shared queues, and repeated H1b observations limit uncertainty estimates. The confidence intervals should not be interpreted as population-level guarantees.

Baseline scope and attribution. The study compares selected local baselines with particular parameter settings, without broad controller tuning or a matched reactive-hold baseline. B2 is an information ablation, not Continuum. The working-set arm is inspired by program pausing, not a faithful ThunderAgent implementation. The no-hold ablation measures the effect of holds within M2; it cannot isolate the value of the forecast from all alternative hold strategies. The event-heap lookahead arms neither optimize a common objective nor reveal all counterfactual future arrivals.

Required validation. A hardware study should compare native priority, rules, forecast policies, matched reactive holds, and published implementations on a common workload, reporting latency, completion time, task quality, and cache costs. The repository proposes a two-H100 Dynamo setup with instrumented background tools and interactive agents. Before such a study, paired simulation/hardware calibration, more seeds, signature-aware progress evaluation, and direct attribution of forecast and index effects would clarify which findings transfer. Both the fixed-workload cap ablation and revised-rule reruns have been completed (Sections 6.5–6.6). Section 8 specifies the remaining validation and deployment work.

8 Future Work and Technical Validation Plan

The experiments establish an information advantage more clearly than a control advantage. The next phase must therefore identify which decisions are sensitive to that information before extending deployment scale. Figure 20 orders the main dependencies. This section distinguishes implemented mechanisms awaiting evaluation from proposed extensions; the latter equations specify candidate designs, not algorithms validated by the reported results.

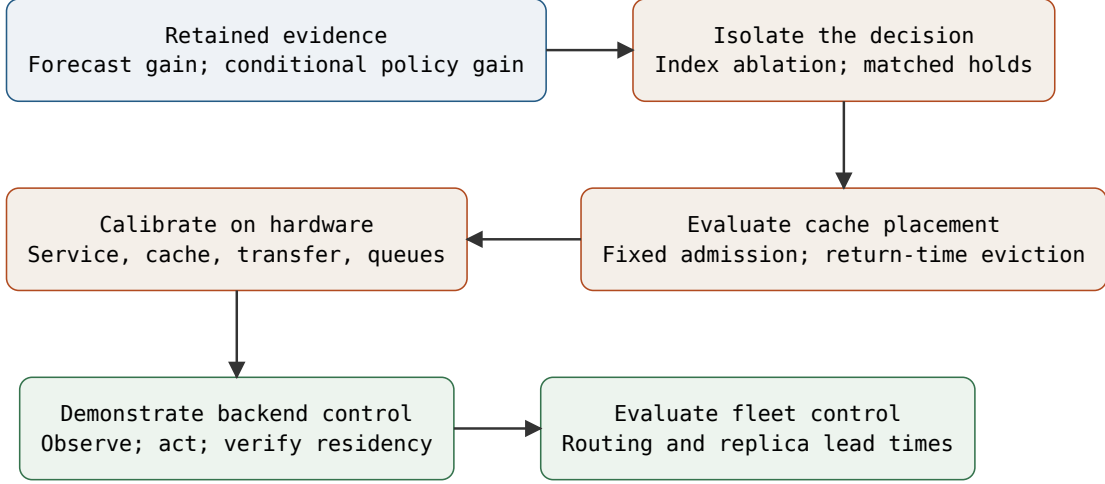


Figure 20: Technical dependencies for the remaining study. Controller attribution and cache-policy evaluation precede claims about hardware benefit. Explicit tier placement requires verified backend control; fleet-level replica experiments additionally require measured capacity and provisioning lead times. Forecast calibration and workload expansion apply throughout.

8.1 Isolate forecast value from admission and ordering

GDP-lite v2 and its two completed reruns are reported in Section 6.6; repeating the original cap ablation is no longer the next step. The outstanding attribution experiment is the implemented `oracle_rule_noidx` arm, configured in `experiments/h2sim_interactive_long_idx.yaml`. It retains the same-rule lookahead hold decision while setting the next-tool contribution in Equation (19) to zero. Pairing it with `oracle_rule` on identical programs isolates the ordering change more closely than comparing either with native. Completion time, deadline attainment, per-session wait, and the distribution of consecutive bypasses should be recorded to test whether the exact-duration index disadvantages particular background programs. No completed score for this diagnostic is included in the present artifact.

A second required control is a reactive policy matched to the forecast policy’s imposed-delay budget. Candidate policies should share class tiers, admission window, cap, and engine placement, while varying only the signal used to trigger holds. Matching mean hold time alone is insufficient: report its distribution, the number of distinct sessions delayed, held KV, and queueing after cap expiry. Sweep the cap and index coefficient β on training workloads, then lock them for evaluation. Compare policies at common background-cost budgets rather than selecting a favorable latency point after observing the test results.

For two policies a, b , a direct paired effect for per-session outcome Y_j is

$$\hat{\Delta}_{a,b} = \frac{1}{|\mathcal{J}_{ab}|} \sum_{j \in \mathcal{J}_{ab}} (Y_j^a - Y_j^b), \quad (30)$$

where $\mathcal{J}_{ab}$ contains session identifiers present in both arms. Outcomes should be chosen separately for

interactive attainment and background JCT. More independent workload seeds are needed; a hierarchical analysis can resample seeds first and whole session families within seeds. This would address more sources of variation than the current pooled-session bootstrap, although shared-queue dependence still requires care.

8.2 Align predicted demand with time-varying capacity

The v2 capacity-release estimate reduces unnecessary holds, but adding capacity freed during a slot still does not model the timing of arrivals and departures within it. A stronger controller needs occupancy trajectories rather than only cumulative first-return demand. One proposed representation is

$$O_r^{(m)}(u) = O_{r,0}^{(m)}(u) + \sum_k a_{kr}^{(m)} \mathbf{1}[A_k^{(m)} \leq u < F_k^{(m)}], \quad u \in [t, t+h], \quad (31)$$

where r identifies a resource, m a sampled future, $O_{r,0}$ the remaining occupancy of work already running, and A_k, F_k, a_{kr} the admission time, release time, and resource use of future request k . For batch slots, $a_{kr} = 1$; for KV, residency must instead follow allocation, growth, sharing, eviction, and offload events, which can extend beyond request completion. A separate cache-state process is consequently required for memory.

This model must include repeated agent turns, service-time uncertainty, and feedback from the proposed admission schedule. A candidate overload constraint on decision-grid times $\mathcal{G}$ is

$$\frac{1}{M} \sum_{m=1}^M \mathbf{1} \left[\max_{u \in \mathcal{G}} \{O_r^{(m)}(u) - C_r(u)\} > 0 \right] \leq \epsilon_r, \quad (32)$$

where $C_r(u)$ is available resource capacity and ϵ_r an allowed empirical violation fraction. It is a sample-based design constraint, not a guaranteed real-world probability bound. A finite grid can miss between-grid peaks, and miscalibrated samples can understate risk. Validate realized overflow and intervention rates on held-out surges. If constraints are imposed separately on K slots, a per-slot error allowance ϵ provides only the union-bound allowance $K\epsilon$ for any-slot failure; horizon-wide risk must be budgeted explicitly.

8.3 Evaluate return-aware KV eviction and demonstrate tier placement

The repository implements `forecast_M1_kv`, `forecast_M2_kv`, and `oracle_kv` in `src/atfm/sim/kv_placement.py`, with loaded and long-interactive configurations. These arms use rules-only admission and change the eviction order among eligible idle sessions. The forecast versions estimate an absolute return time from the median of finite sampled return times; an unavailable estimate is treated as infinity. The policy evicts the latest predicted return first. This is implemented simulator functionality, but completed results are not included here.

A minimal comparison must hold the request order fixed while comparing LRU, M1, M2, and lookahead eviction. Record prefix tokens recomputed, TTFT after tool return, cache hits, background JCT, and victims per eviction. Vary context size, KV capacity, tool-duration heterogeneity, progress availability, and return-time correlation. The current median-of-finite-samples rule also discards the probability of never returning when at least one finite sample exists; test whether retaining that probability improves decisions. Lookahead provides an information diagnostic, not an optimal cache policy under variable sizes and repeated requests.

Explicit HBM/DRAM/SSD placement is a separate step. First demonstrate that a backend can identify a session's resident prefix, demote or promote it, and verify the resulting residency and resumption cost. Unsupported operations should remain logged advisory directives. Measure transfer latency as a function of prefix size and concurrent transfers; do not assume that bandwidth alone determines completion. For a prefix of size V_i bytes, an initial approximation is

$$\ell_i^{\text{move}} = \ell_0 + \frac{V_i}{B}, \quad t_i^{\text{start}} = t + q_\alpha(R_i(t)) - \ell_i^{\text{move}} - \delta, \quad (33)$$

where ℓ_0 is setup latency, $\widehat{B}$ effective transfer bandwidth in bytes/s, δ a safety margin, and $q_\alpha(R_i(t))$ an early return quantile. A start time already in the past requests immediate action, subject to capacity. Under an accurate continuous return distribution, an early quantile targets roughly an α probability of return before the planned ready time; actual success also depends on transfer uncertainty. Evaluate missed pre-staging deadlines, unused transfers, memory pressure, and interference with model execution before claiming benefit.

8.4 Improve calibration, progress generalization, and event integrity

The signature-aware progress implementation must be evaluated with new held-out collections rather than credited with historical H1b scores. Split by command signature and environment, including build parallelism, input size, cache warmth, and machine load. Retain all observations from a tool execution in one split. Compare M1, pooled M2, signature-aware M2, and online adaptation; report per-family errors and pooled errors so that long jobs do not conceal short-job failures. Silent tools and sparse, stale, reset, or out-of-order counters require explicit fallback tests.

Fleet evaluation should add correlated arrivals, shared-backend slowdowns, multi-level fan-out, and changing fractions of exogenous demand. M3’s shared slowdown factor needs a controlled comparison with the same marginal duration distributions but different cross-session dependence. Calibration should be fitted on disjoint time blocks and assessed by class, horizon, workload, and load regime. Report coverage, interval width, pinball loss, and CRPS together. Track $\Pr(R_i = \infty)$, uncertain tool-output size, and pending gaps separately; widening intervals alone does not correct biased return probabilities.

Deployment also requires validating the event path. The implemented in-memory and JSONL buses should be complemented by a durable deployment transport, with event identifiers, per-session sequence numbers, replay checkpoints, and bounded staleness. Duplicate delivery must not double-count a child or a request; late completion must reconcile stale running state. Measure event-to-forecast lag, missed-event rate, board CPU cost, and memory per active session. Board loss should cause bounded fallback and hold expiry, with no dependence on a fresh forecast to release an expired hold. These checks establish operational behavior; they do not substitute for predictive validation.

8.5 Validate the simulator and controller on GPUs

The repository’s proposed hardware sequence starts with a single GPU to characterize service and cache behavior, then two H100 GPUs for the main study. Two independent replicas permit routing experiments; a model requiring two-way tensor parallelism instead tests a single distributed replica. The deployment choice must therefore follow the question being tested. Freeze model weights, tokenizer, quantization, serving-engine revision, scheduling settings, and trace inputs for every paired comparison.

Replace fixed prefill/decode rates with measured profiles conditioned on prompt length, output length, batch size, and cache-hit fraction. Measure transfer and recomputation separately, and verify whether proxy priority reaches the engine queue. Run paired simulator/hardware cells spanning underload, sustained contention, interactive surges, synchronized tool completions, and worker loss. For metric z , report relative discrepancy

$$E_z = \frac{|z_{\text{sim}} - z_{\text{hardware}}|}{\max(|z_{\text{hardware}}|, \eta_z)}, \quad (34)$$

where $\eta_z > 0$ is a declared, metric-specific small denominator floor. Include mean and tail JCT, post-tool TTFT, cache-miss volume, and the ordering of policy effects; accurate averages alone are inadequate. The project plan proposes a 6% JCT-error target, which is a prospective validation target rather than an achieved result or a universal threshold.

The policy study should include native engine priority, rules-only admission, matched reactive holds, forecast policies, and compatible published implementations under common resource and tuning budgets. Report task success or quality, interactive attainment, background completion and deadlines, GPU-hours, queue location, and cache costs. Preserve paired programs where possible, but also run live agents:

admission delays may change external tool outcomes or agent trajectories in ways fixed trace replay cannot capture.

8.6 Extend to routing and replica control only after calibration

The architecture proposes a planner integration returning a minimum replica floor. This remains future work and should first be exercised with virtual workers. Let ℓ be the measured scale-out lead time, $\widehat{\lambda}_{\text{req}}(t + \ell)$ a forecast request rate at readiness, μ a sustainable per-replica rate for the relevant workload, and $\rho_* < 1$ a target utilization. A first approximation is

$$n_{\min}(t) = \left\lceil \frac{q_{0.9}(\widehat{\lambda}_{\text{req}}(t + \ell))}{\rho_* \mu} \right\rceil. \quad (35)$$

The rate forecast in this equation is an extension of the present first-return target: it must include repeated requests and map their sizes to service demand. Heterogeneous replicas, routing affinity, KV capacity, prefill/decode bottlenecks, and uncertain startup times require a richer feasibility model. Apply warm-up accounting, scale-down hysteresis, and minimum dwell times; otherwise forecast noise can produce oscillation.

Evaluate reactive scaling against the predictive floor at matched SLO or GPU-hour budgets, including cold starts and idle reservations in cost. A hybrid setup with two real workers and additional Mocker workers can test integration and sensitivity at larger logical scale, but cannot establish real multi-node transfer cost or real provisioning lead times. Those claims require physical multi-node experiments. Success at each stage should be a reproducible improvement in the latency–cost trade-off under held-out workloads, not merely a lower forecast loss or a functioning control API.

9 Conclusion

ATFM treats external agent execution as observable state for inference-demand forecasting. Across two coding-agent corpora, elapsed-time conditioning improves medium-horizon first-resumption forecasts over an aggregate-history baseline. Live progress further improves remaining-time scores on selected long-running tools, with identifiable failures when progress curves differ between training and evaluation. Closed-loop results are negative for the admission lever as implemented: forecast-based holds improve interactive SLO attainment only when the cap permits ten-minute deferrals, a controlled cap ablation removes the gain, and true future demand does not restore it, while rules-only admission gives a small consistent gain at little cost. Revised capacity accounting reduces the cost of holds but still does not establish an SLO gain. The principal finding is that session state is useful for forecasting, while converting that information into a favorable serving policy remains workload- and controller-dependent.

References

- [1] Hanchen Li et al. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv:2511.02230, 2025. URL <https://arxiv.org/abs/2511.02230>. Revised 2026.
- [2] Hao Kang et al. ThunderAgent: A simple, fast and program-aware agentic inference system. arXiv:2602.13692, 2026. URL <https://arxiv.org/abs/2602.13692>.
- [3] Yipeng Liu, Yingqiang Zhang, Feifei Li, and Huanchen Zhang. Ask the tool, don’t guess: Agent tool calls hold their progress, and the serving system should read it. arXiv:2609.18849, 2026. URL <https://arxiv.org/abs/2609.18849>.

- [4] Jianru Ding, Ryien Hosseini, Pouya Mahdi Gholami, Mingyuan Xiang, and Henry Hoffmann. Observation, not prediction: Conversation-level disaggregated scheduling for agentic serving. arXiv:2606.01839, 2026. URL <https://arxiv.org/abs/2606.01839>.
- [5] NVIDIA. Full-stack optimizations for agentic inference with Dynamo. NVIDIA Dynamo documentation, 2026. URL <https://docs.nvidia.com/dynamo/dev/digest/agentic-inference>. Accessed September 27, 2026.
- [6] Jialiang Huang et al. DeepSeek Elastic Compute (DSec): A sandbox infrastructure for effective agentic training at scale. arXiv:2609.22978, 2026. URL <https://arxiv.org/abs/2609.22978>.
- [7] UW SyFI. TraceLab: Coding-agent trace dataset and collection toolkit, 2026. URL <https://github.com/uw-syfi/TraceLab>. See also *TraceLab: Characterizing Coding Agent Workloads for LLM Serving*, arXiv:2606.30560.
- [8] SemiAnalysis. Claude code traces with subagents (cc-traces-weka-062126). Hugging Face dataset used by AgentX, 2026. URL <https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126>.
- [9] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

A Artifact and Reproducibility

The editable manuscript is `docs/paper/atfm-paper.tex`, with BibTeX references in `references.bib` and vector PDF figures in `latex-fig/`. Tables and figures retain the values in the frozen CSVs under `docs/paper/results/`. The LaTeX document builds directly with `latexmk`; the optional `build_latex.py` wrapper checks compilation and publishes the PDF. Its `-figures` option refreshes vector figures from the saved result summaries. That directory includes a manifest with original run paths, source hashes, and the repository commit at capture. Raw `data/` and `runs/` directories are ignored by Git, so retaining these summaries makes paper rebuilding independent of locally collected raw traces. It does not by itself reproduce the experiments.

The repository specifies Python 3.12 and locks dependencies with `uv.lock`. The principal implementation paths are `src/atfm/board/` for session models and aggregation, `sidecar/` for tool observation, `proxy/` for admission, `sim/` for closed-loop execution, and `eval/` for scores and bootstrap metrics. Experiment configurations reside in `experiments/`. The architecture specification and dated research notes in `docs/` provide design history; when a narrative note disagrees with retained metrics, this report uses the metric files.

Table A1: Provenance of the main results. Paths are relative to the repository root; the corresponding summaries are frozen with this paper.

Result	Source artifact
H1a original run	<code>runs/h1_tracelab_r200/metrics.csv</code>
H1a sweeps	<code>runs/sweep_tracelab.csv</code> ; <code>runs/sweep_agentx.csv</code>
H1b retained collection	<code>runs/h1b_with_numpy.csv</code> , aggregated by family/model and by model
Loaded H2	<code>runs/h2sim_long_tool_loaded/{metrics,paired}.csv</code>
Long-interactive H2	<code>runs/h2sim_interactive_long/{metrics,paired}.csv</code>
Loaded H2, 60-second cap	<code>runs/h2sim_long_tool_loaded_cap60/{metrics,paired}.csv</code>
GDP-lite v2 reruns	<code>runs/h2sim_interactive_long_v2/{metrics,paired}.csv</code> ; <code>runs/h2sim_long_tool_loaded_cap60_v2/{metrics,paired}.csv</code>

From the repository root, the following commands rebuild the document and show the experiment entry points. Experiment commands require their configured raw inputs and may be expensive; they are distinct from the paper build.

```
python3 docs/paper/build_latex.py
uv run python scripts/run_h1.py experiments/h1_tracelab.yaml
uv run python scripts/sweep_h1.py experiments/h1_agentx.yaml \
  --seeds 0 1 2 --rates 100 200 400 --out runs/sweep_agentx.csv
uv run python scripts/h1b_resumption.py <collection-events.jsonl> \
  --min-duration 30 --offset 15 --out <scores.csv>
uv run python scripts/run_h2sim.py experiments/h2sim_loaded.yaml
uv run python scripts/run_h2sim.py experiments/h2sim_interactive_long.yaml
uv run python scripts/run_h2sim.py experiments/h2sim_loaded_cap60.yaml
uv run python scripts/run_h2sim.py experiments/h2sim_loaded_cap60_v2.yaml
uv run python scripts/run_h2sim.py experiments/h2sim_interactive_long_v2.yaml
```

The reported H1b aggregate combines the earlier build/pipeline collections with NumPy runs; recreating it requires the same collection composition and historical predictor implementation. The current command-signature feature can change rescored values. Earlier exploratory results and integration checks are documented in `docs/research/2026-09-22-h1-first-results.md`, `2026-09-23-h1-second-corpus-and-h1b.md`, `2026-09-23-l1-results.md`, and `2026-09-24-h2sim-first-results.md`. The completed loaded 60-second-cap experiment is reported in Section 6.5; the two completed GDP-lite v2 reruns are reported in Section 6.6. KV-placement and index-diagnostic outcomes remain outside this evidence snapshot.

Writing assistance. Earlier project materials disclosed assistance from Claude Code. This revision used Codex for editorial revision, consistency checks, and document generation; retained experiment summaries are identified above.

B Additional Loaded-Regime Costs

Figure 21 shows the resource costs and their spread across the three seeds.

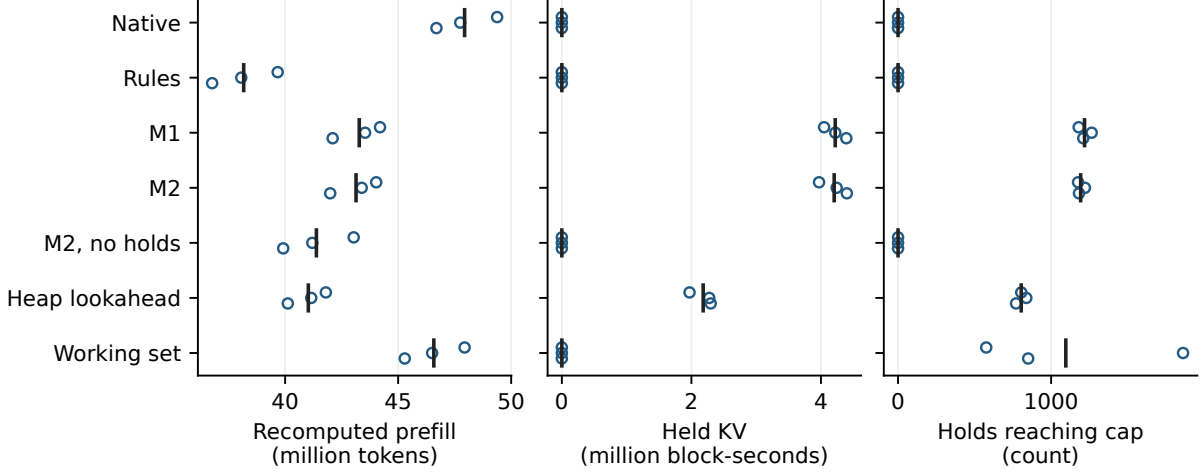


Figure 21: Additional costs in the loaded 600-second-cap regime. Open circles are the three seed values; dark vertical marks are their arithmetic means. Lower values indicate less recomputation, fewer retained block-seconds during holds, or fewer cap hits, respectively. These metrics measure distinct costs and are not combined into a single score. Zero held-KV accounting for the working-set arm does not imply zero delay, since paused state can be evicted.

Table B1: Loaded long-tool regime. TTFT is the arithmetic mean of per-seed p95 values, not the p95 of pooled calls. Deadline attainment is pooled over common sessions. Recomputed prefill and held-cache occupancy are in millions per seed; cap counts are means per seed.

Arm	p95 TTFT (s)	Deadline (%)	Recompute (M tokens)	Held KV (M block-s)	At cap
Native	2.67	19.0	47.94	0.00	0
Rules	3.01	24.7	38.17	0.00	0
Forecast M1	2.16	11.9	43.28	4.22	1,218
Forecast M2	2.12	12.2	43.14	4.20	1,193
M2, no holds	3.23	24.3	41.38	0.00	0
Heap lookahead	2.51	9.1	41.03	2.18	805
Working set	2.29	11.4	46.58	0.00	1,096

Recomputed prefill counts work after eviction. Held KV integrates resident memory during a policy hold; zero held-KV accounting for the working-set arm does not imply zero delay, since paused state can be evicted. Cap counts do not bound subsequent window queueing.

In particular, held-cache accounting is an integral,

$$C_{\text{hold}} = \int_0^{T_{\text{run}}} \sum_i B_i^{\text{resident}}(u) \mathbf{1}[i \text{ is policy-held at } u] du, \quad (36)$$

where T_{run} is the simulation end time and $B_i^{\text{resident}}(u)$ is session i 's retained block count at time u . Its units are block-seconds: retaining 100 blocks for 20 seconds contributes 2,000 block-seconds. This quantity is neither peak memory nor total waiting time.

C Mathematical Correspondence to Code and Plan

The mathematical additions serve three purposes: specify implemented transformations, derive consequences of their assumptions, and formalize outstanding work in the research plan. They do not introduce additional measured algorithms. Tables C1 and C2 make these roles explicit. Code paths below are relative to `src/atfm/`; plan references are to `docs/superpowers/specs/2026-09-22-atfm-architecture-design.md`, unless another document is named.

Table C1: Correspondence between the mathematical model and the implemented forecast/evaluation path. A derived explanation is not an additional model used in the experiments.

Mathematical element	Implementation and scope
First-return sum and Monte Carlo CDF (Equations (1), (14))	<code>board/forecaster.py</code> : <code>SessionForecaster.forecast</code> ; independently sampled next-call size and return time per session, with optional shared backend factors. Implemented aggregation; analytic moments explain its assumptions.
Survival conditioning (Equations (3), (4))	<code>board/predictors/duration.py</code> : <code>DurationModel.sample_conditional</code> and <code>SurvivalPredictor</code> . Sparse surviving histories invoke a tail fallback, so the empirical CDF is not the entire algorithm. Weibull/hazard figures are explanatory examples of the same conditioning operation.
Return mixture and pending gap (Equations (6), (7))	<code>board/predictors/duration.py</code> : <code>HistoryPredictor._finish</code> adds a sampled gap and no-return mass for tool-running states. <code>_off_tool</code> uses separate model-running and pending-phase rules. No-return frequency is estimated from observed last turns; trace truncation can confound termination.
Progress curve and Gamma rate update (Equations (5), (11))	<code>board/predictors/progress.py</code> : <code>ProgressCurve</code> , <code>rate_posterior</code> , and <code>ProgressPredictor._remaining_from_progress</code> . Curve availability determines the branch. Default prior-rate estimation, sampled curve noise, residuals, and report age distinguish the implementation from the idealized derivation.
Exogenous compound demand (Equation (16))	<code>board/forecaster.py</code> : <code>ExogenousModel.draw</code> ; plug-in Poisson rate and empirical first-call sizes. Rate uncertainty in Equation (17) and cross-horizon coupling are extensions, not implemented sampling features.
Dependence (Equation (15))	General variance identity; <code>board/predictors/backend.py</code> and the forecaster’s shared factor supply the exploratory M3 mechanism. The two-session example diagnoses dependence sensitivity; it is not a fitted correlation estimate.
Calibration (Equation (28))	<code>board/calibrate.py</code> : <code>inflate</code> applies centering, widening and zero-clipping; <code>fit_inflation</code> selects from a grid using training-held-out coverage. No distribution-free guarantee is implemented.
Scores and sample precision	<code>eval/forecast.py</code> : sample CRPS, interpolated NumPy quantiles, pinball loss and coverage. Equation (25) is an idealized independent-draw error calculation, not the experiment’s confidence-interval estimator.

The CDF inverse in Equation (14) defines the conceptual quantile; executable scoring and hold decisions use NumPy’s interpolated sample quantile, so finite-sample values need not equal an observed draw. For finite forecast samples $x_1, \dots, x_M$, the implemented empirical-distribution CRPS is

$$\widehat{\text{CRPS}}(y) = \frac{1}{M} \sum_m |x_m - y| - \frac{1}{2M^2} \sum_{m=1}^M \sum_{n=1}^M |x_m - x_n|. \quad (37)$$

The evaluator computes the pairwise term efficiently by sorting. It replaces nonfinite samples by a 10^{12} sentinel when scoring. Consequently, scores for distributions with no-return mass depend on this finite convention; the continuous finite-moment CRPS formula must not be read as a justification for an infinite-duration score. In H1a, a non-returning session contributes zero to finite-horizon demand, so this does not make fleet-demand samples infinite.

The controller extensions below follow the existing delivery plan. Their mathematical form makes the required resource model and validation boundary explicit.

Table C2: Connection of the controller mathematics and outstanding work to the repository’s technical plan. The equations specify candidate models or diagnostics where implementation is incomplete.

Element	Code, plan, and validation boundary
Ordering and original/v2 holds	<code>proxy/index.py</code> and <code>sim/forecast_arm.py</code> : <code>index</code> , <code>GdpLite</code> , and <code>freeing_capacity</code> . Architecture Sections 6.1–6.2 motivate tactical ordering and demand-based delay; the implemented single-slot heuristic is simpler than the proposed multi-slot controller.
Occupancy and overload (Equations (31), (32))	Proposed refinement of the architecture’s Section 6.2 resource constraints, prompted by the completed cap and v2 diagnostics. Requires coherent future paths, cache-residency dynamics, and out-of-sample overflow validation; these equations are not evaluated policy code.
Index attribution	<code>experiments/h2sim.py</code> : <code>oracle_rule_noidx</code> arm construction; <code>experiments/h2sim_interactive_long_idx.yaml</code> relative to the repository root. Motivated by the v2 research note’s residual lookahead cost. No result is claimed from the diagnostic’s existence.
KV eviction and tier placement	<code>sim/kv_placement.py</code> implements eviction ordering. Architecture Section 6.3 and decision D3 defer explicit tier actions until a backend mechanism is demonstrated. Equation (33) formalizes that proposed lead-time calculation, not an existing transfer service.
Calibration and generalization	Architecture Sections 5 and 9, and <code>docs/research/2026-09-23-h1-second-corpus-and-h1b.md</code> , motivate held-out progress signatures, correlated slowdowns, and calibration. Fixed-prior plots, uncertain-rate moments, and coherent arrival paths identify assumptions to test; they add no new benchmark outcomes.
Hardware and replica floor	Architecture decision D8, Section 6.4, and <code>docs/research/2026-09-22-demonstration-ladder.md</code> define the hardware sequence and virtual-worker stage. Equation (35) is a proposed workload-rate approximation requiring measured per-replica capacity and startup time.